

Performance Analysis

[Reattempt](#)[Download Certificate](#)

Detailed analysis of your performance in the AWS Certified Solutions Architect Associate - SAA C03 exam.

Answer Distribution

0

Correct

0

Incorrect

65

Unattempted



ITcertifications



Domain Performance

Storage	0%
Serverless	0%
Security	0%
Compute	0%
Databases	0%
Networking	0%
Automation	0%
Monitoring and Observability	0%

Final Score

**0/65**

Passing score: 48

Time Taken

**130m 0s**

Total time: 130 minutes

Question Analysis

✓ Correct	0
✗ Incorrect	0
⊖ Unattempted	65
🔖 Reviewed	0

Question-wise Analysis

Expand All

Collapse All

Question 1

Not Attempted

Storage



A company collects data for temperature, humidity, and atmospheric pressure in cities across multiple continents. The average volume of data that the company collects from each site daily is 500 GB. Each site has a high-speed Internet connection. The company wants to aggregate the data from all these global sites as quickly as possible in a single Amazon S3 bucket. The solution must minimize operational complexity. Which solution meets these requirements?

- ✓ Turn on S3 Transfer Acceleration on the destination S3 bucket. Use multipart uploads to directly upload site data to the destination S3 bucket.

S3 Transfer Acceleration is designed for fast, easy, and secure transfers of files over long distances. Multipart uploads allow large files to be uploaded in parallel, improving speed. This option minimizes operational complexity by directly uploading to the destination bucket.

- ✗ Upload the data from each site to an S3 bucket in the closest Region. Use S3 Cross-Region Replication to copy objects to the destination S3 bucket. Then remove the data from the origin S3 bucket.

While this option uses the closest region for upload, Cross-Region Replication adds operational complexity and extra costs compared to direct upload with Transfer Acceleration. Removing data also adds an extra step.

- ✗ Schedule AWS Snowball Edge Storage Optimized device jobs daily to transfer data from each site to the closest Region. Use S3 Cross-Region Replication to copy objects to the destination S3 bucket.

Using Snowball Edge is unnecessary since each site has a high-speed Internet connection. This adds significant operational complexity and cost. Also, Cross-Region replication is also used, adding to the complexity.

- ✗ Upload the data from each site to an Amazon EC2 instance in the closest Region. Store the data in an Amazon Elastic Block Store (Amazon EBS) volume. At regular intervals, take an EBS snapshot and copy it to the Region that contains the destination S3 bucket. Restore the EBS volume in that Region.

This solution introduces significant operational complexity by requiring EC2 instances, EBS volumes, snapshots, and restoration. It's far more complex than direct upload to S3 using Transfer Acceleration.

Overall Explanation:

S3 Transfer Acceleration with multipart uploads provides the fastest and least complex solution for aggregating data from multiple global sites into a single S3 bucket. It leverages the existing high-speed internet connections at each site and avoids the overhead of intermediate storage or complex replication setups. Options B, C, and D are more complex and less efficient. S3 Transfer Acceleration optimizes the transfer path by using edge locations and provides faster upload speeds, especially for large files using multipart uploads.

 View Discussions

 Ask AI

Question 2

Not Attempted

Serverless



A company is developing an e-commerce application on AWS. New order information is sent to an Amazon API Gateway REST API for processing. To guarantee that orders are processed in the sequence they are received, which solution should the company implement?

- ☐ Use an API Gateway integration to publish a message to an Amazon Simple Notification Service (Amazon SNS) topic when the application receives an order. Subscribe an AWS Lambda function to the topic to perform processing.

SNS is a publish-subscribe service and does not guarantee message order. SNS is suited for fan-out scenarios and notifications, not ordered processing.

- ☒ Use an API Gateway integration to send a message to an Amazon Simple Queue Service (Amazon SQS) FIFO queue when the application receives an order. Configure the SQS FIFO queue to invoke an AWS Lambda function for processing.

SQS FIFO queues are designed to maintain message order, ensuring that messages are processed in the exact sequence they were sent. This is the correct solution for ensuring orders are processed in the order they are received.

- ☐ Use an API Gateway authorizer to block any requests while the application processes an order.

API Gateway authorizers are used for authentication and authorization, not for managing message order. Blocking requests would lead to a poor user experience and does not guarantee ordered processing.

- ☐ Use an API Gateway integration to send a message to an Amazon Simple Queue Service (Amazon SQS) standard queue when the application receives an order. Configure the SQS standard queue to invoke an AWS Lambda function for processing.

SQS standard queues do not guarantee message order. While they offer at-least-once delivery, messages may occasionally arrive out of order, making them unsuitable for this requirement.

Overall Explanation:

The correct solution is to use an Amazon SQS FIFO queue. FIFO (First-In-First-Out) queues guarantee that messages are processed in the order they are received. The API Gateway sends order information to the SQS FIFO queue, and a Lambda function triggered by the queue processes the orders in the correct sequence. SNS does not guarantee order, and standard SQS queues also do not preserve order. API Gateway authorizers are irrelevant to the sequencing requirement.

 View Discussions Ask AI

Question 3

Not Attempted

Security 

A company's containerized application runs on an Amazon EC2 instance. The application needs to download security certificates before it can communicate with other business applications. The company wants a highly secure solution to encrypt and decrypt the certificates in near real time. The solution also needs to store data in highly available storage after the data is encrypted. Which solution will meet these requirements with the LEAST operational overhead?

- ☐ Create AWS Secrets Manager secrets for encrypted certificates. Manually update the certificates as needed. Control access to the data by using fine-grained IAM access.
This option requires manual updates, increasing operational overhead. Also, it doesn't specify how to store the encrypted data in a highly available manner.
- ☐ Create an AWS Lambda function that uses the Python cryptography library to receive and perform encryption operations. Store the function in an Amazon S3 bucket.
Using a Lambda function and storing it in S3 doesn't directly address the encryption key management or the storage of the *encrypted data* itself in a highly available manner. It also adds operational complexity.
- ☒ Create an AWS Key Management Service (AWS KMS) customer managed key. Allow the EC2 role to use the KMS key for encryption operations. Store the encrypted data on Amazon S3.
This option leverages AWS KMS for secure and managed encryption/decryption. S3 provides highly available storage. Granting the EC2 role permission simplifies access and reduces overhead.
- ☐ Create an AWS Key Management Service (AWS KMS) customer managed key. Allow the EC2 role to use the KMS key for encryption operations. Store the encrypted data on Amazon Elastic Block Store (Amazon EBS) volumes.
While this uses KMS for encryption, EBS isn't as inherently highly available as S3, and managing EBS volumes can add operational overhead compared to S3.

Overall Explanation:

Option C is the best solution. AWS KMS offers a managed service for encryption, minimizing operational overhead. Storing the encrypted data on Amazon S3 ensures high availability. Providing the EC2 role access to the KMS key eliminates the need for managing individual credentials. S3 provides durability by redundantly storing the data across

multiple Availability Zones. EBS provides durability by redundantly storing the data in a single Availability Zone. While EBS has high availability, S3's inherent design for object storage makes it a better fit for this scenario. S3 is accessed via APIs, EBS is attached to a single instance.

[View Discussions](#)[Ask AI](#)

Question 4

Not Attempted

Storage



A company currently stores 5 TB of data in on-premises block storage systems. The company's current storage solution provides limited space for additional data. The company runs applications on-premises that must be able to retrieve frequently accessed data with low latency. The company requires a cloud-based storage solution. Which solution will meet these requirements with the MOST operational efficiency?

- ☐ Use Amazon S3 File Gateway. Integrate S3 File Gateway with the on-premises applications to store and directly retrieve files by using the SMB file system.
S3 File Gateway is designed for file-based storage and may not be the most efficient solution for applications requiring block storage with low latency.
- ☒ Use an AWS Storage Gateway Volume Gateway with cached volumes as iSCSI targets.
Volume Gateway with cached volumes stores only the most frequently accessed data on-premises, while the entire dataset is stored in Amazon S3. This provides low-latency access to frequently accessed data while leveraging the scalability of S3.
- ☐ Use an AWS Storage Gateway Volume Gateway with stored volumes as iSCSI targets.
Volume Gateway with stored volumes stores the entire dataset on-premises and asynchronously backs it up to AWS. This option does not address the company's requirement for limited on-premises storage capacity.
- ☐ Use an AWS Storage Gateway Tape Gateway. Integrate Tape Gateway with the on-premises applications to store virtual tapes in Amazon S3.
Tape Gateway is designed for archival purposes and is not suitable for applications requiring low-latency access to frequently accessed data.

Overall Explanation:

The company needs low-latency access to frequently accessed data with limited on-premise storage. Cached volumes store only the most frequently accessed data on-premises, while the full dataset is stored in Amazon S3. This ensures low-latency access to frequently accessed data while leveraging the scalability and durability of S3 for the full dataset. The other options don't satisfy the need for low latency or limited on-premise storage.

[View Discussions](#)[Ask AI](#)

Question 5

Not Attempted

[Compute](#)

A food delivery company is experiencing scaling issues during peak hours with its order processing system. The system uses EC2 instances in Auto Scaling groups for order collection and fulfillment. Order collection is fast, but fulfillment takes longer. No data loss is acceptable during scaling events. A solutions architect needs to ensure adequate scaling for both processes during peak times. Which solution meets these requirements?

- ⊗ Use Amazon CloudWatch to monitor the CPUUtilization metric for each instance in both Auto Scaling groups. Configure each Auto Scaling group's minimum capacity to meet its peak workload value.

This approach may handle load, but doesn't address potential message buildup or guarantee data durability in the event of instance failures. It relies solely on CPU utilization, which may not accurately reflect the order backlog.

- ⊗ Use Amazon CloudWatch to monitor the CPUUtilization metric for each instance in both Auto Scaling groups. Configure a CloudWatch alarm to invoke an Amazon Simple Notification Service (Amazon SNS) topic to create additional Auto Scaling groups on demand.

Using SNS to trigger scaling is less responsive than directly scaling based on queue depth. Moreover, relying on CPU utilization as a scaling trigger is not as effective as scaling based on the number of pending orders.

- ⊗ Provision two Amazon Simple Queue Service (Amazon SQS) queues. Use one SQS queue for order collection. Use the second SQS queue for order fulfillment. Configure the EC2 instances to poll their respective queues. Scale the Auto Scaling groups based on notifications that the queues send.

Scaling based on queue notifications is vague and doesn't directly correlate to the workload. It also introduces unnecessary complexity. SQS already provides metrics that can be used for scaling.

- ✔ Provision two Amazon Simple Queue Service (Amazon SQS) queues. Use one SQS queue for order collection. Use the second SQS queue for order fulfillment. Configure the EC2 instances to poll their respective queues. Scale the Auto Scaling groups based on the number of messages in each queue.

This is the optimal solution. SQS provides a durable queue for messages. Scaling based on queue depth allows for autoscaling based on the actual backlog of orders, ensuring efficient resource allocation and preventing data loss. EC2 instances pull messages from the queue, processing them and scaling based on demand.

Overall Explanation:

The best solution is to use SQS queues for order collection and fulfillment. This provides decoupling and durability. Scaling the Auto Scaling groups based on the number of messages in the respective queues ensures that resources are dynamically allocated based on the workload, preventing data loss and addressing scaling issues during peak traffic.

 View Discussions

 Ask AI

Question 6

Not Attempted

Databases



An online gaming company is migrating its user data storage to Amazon DynamoDB to accommodate its growing user base. The company's DynamoDB tables store user profiles, achievements, and in-game transactions. The company requires a resilient and continuously available DynamoDB architecture to ensure a seamless gaming experience while remaining cost-effective. Which solution BEST fulfills these requirements?

- ⊗ Create DynamoDB tables in a single AWS Region. Use on-demand capacity mode. Use global tables to replicate data across multiple Regions.

This option uses on-demand capacity mode, which can be expensive for predictable traffic. While global tables provide replication, deploying tables in a single region makes the application susceptible to regional failures.

- ⊗ Use DynamoDB Accelerator (DAX) to cache frequently accessed data. Deploy tables in a single AWS Region and enable auto scaling. Configure Cross-Region Replication manually to additional Regions.

DAX improves read performance but doesn't provide inherent resilience. Manual Cross-Region Replication is complex to manage and less reliable than automated solutions. Deploying tables in a single region makes the application susceptible to regional failures.

- ⊗ Create DynamoDB tables in multiple AWS Regions. Use on-demand capacity mode. Use DynamoDB Streams for Cross-Region Replication between Regions.

DynamoDB Streams require custom logic to handle conflict resolution during replication, making them less robust and more complex to manage than Global Tables. On-demand can be expensive.

- ✔ Use DynamoDB global tables for automatic multi-Region replication. Deploy tables in multiple AWS Regions. Use provisioned capacity mode. Enable auto scaling.

DynamoDB global tables automatically replicate data across multiple Regions, ensuring availability and consistency. Provisioned capacity mode with auto scaling allows for cost optimization by pre-allocating capacity while dynamically adjusting it based on traffic patterns. Multi-Region deployment improves resilience against regional failures.

Overall Explanation:

The most cost-effective and robust solution is to use DynamoDB global tables with provisioned capacity and auto scaling, deployed across multiple AWS Regions. Global tables ensure automatic, multi-Region replication, providing high availability and disaster recovery capabilities. Provisioned capacity with auto scaling allows for efficient cost management by pre-allocating and dynamically adjusting capacity based on demand. DynamoDB streams require custom conflict resolution logic, and on-demand capacity can become more expensive than provisioned capacity with auto-scaling for predictable workloads.

 View Discussions

 Ask AI

Question 7

Not Attempted

Storage



A company runs its media rendering application on premises. The company wants to reduce storage costs and has moved all data to Amazon S3. The on-premises rendering application needs low-latency access to storage to maintain performance. Which storage solution will meet these requirements in the MOST cost-effective way?

- ⊗ Use Mountpoint for Amazon S3 to access the data in Amazon S3 for the on-premises application.

Mountpoint is not designed for on-premises environments or low-latency access; it works well for cloud-based applications.

- ✔ Configure an Amazon S3 File Gateway to provide storage for the on-premises application.

Amazon S3 File Gateway provides low-latency, on-premises access to data stored in Amazon S3 through a file-based interface. It caches frequently accessed data locally to reduce latency.

- ⊗ Copy the data from Amazon S3 to Amazon FSx for Windows File Server. Configure an Amazon FSx File Gateway to provide storage for the on-premises application.

Copying data to FSx increases costs, as FSx storage is more expensive than S3. It also adds unnecessary complexity.

- ⊗ Configure an on-premises file server. Use the Amazon S3 API to connect to S3 storage. Configure the application to access the storage from the on-premises file server.

Direct interaction with S3 introduces higher latency for on-premises applications and lacks local caching.

Overall Explanation:

Amazon S3 File Gateway provides a cost-effective way to provide low-latency access to data stored in S3 for on-premises applications. It caches frequently accessed data locally, ensuring the required application performance. Mountpoint is not suitable for on-premises environments. Copying the data to FSx increases costs. Direct interaction with S3 lacks local caching and introduces higher latency. Therefore, Amazon S3 File Gateway is the most cost-effective solution.

 View Discussions

 Ask AI

Question 8

Not Attempted

Networking



A company hosts its enterprise resource planning (ERP) system in the us-east-1 Region. The system runs on Amazon EC2 instances. Customers use a public API that is hosted on the EC2 instances to exchange information with the ERP system. International customers report slow API

response times from their data centers. Which solution will improve response times for the international customers MOST cost-effectively?

- ☐ Create an AWS Direct Connect connection that has a public virtual interface (VIF) to provide connectivity from each customer's data center to us-east-1. Route customer API requests by using a Direct Connect gateway to the ERP system API.
Direct Connect requires dedicated connections for each customer, making it expensive and impractical for a public API.
- ☐ Set up an Amazon CloudFront distribution in front of the API. Configure the CachingOptimized managed cache policy to provide improved cache efficiency.
CloudFront improves performance with caching, but its efficiency depends on the API's use case. Dynamic & personalized API responses may not benefit significantly from caching. It's best suited for static content or APIs with predictable responses. For dynamic API interaction with an ERP, caching may not be suitable.
- ☒ Set up AWS Global Accelerator. Configure listeners for the necessary ports. Configure endpoint groups for the appropriate Regions to distribute traffic. Create an endpoint in the group for the API.
AWS Global Accelerator provides a global static IP address and uses the AWS global network to route traffic optimally, reducing latency for geographically distant users. Unlike CloudFront, it doesn't rely on caching contents, making it effective for dynamic content.
- ☐ Use AWS Site-to-Site VPN to establish dedicated VPN tunnels between Regions and customer networks. Route traffic to the API over the VPN connections.
VPNs are not ideal for public APIs because they require specific setup for each customer, resulting in high operational overhead.

Overall Explanation:

AWS Global Accelerator is the most cost-effective solution for improving API response times for international users. It leverages the AWS global network to optimize traffic routing. Unlike CloudFront, it does not rely on caching, which is essential for dynamic API interactions with an ERP system. Direct Connect is too expensive, and VPNs are not suitable for public APIs due to their management overhead. CloudFront might not be efficient for dynamic content.

[View Discussions](#)[Ask AI](#)

Question 9

Not Attempted

Automation



A company hosts customer satisfaction surveys on its website, receiving thousands of responses hourly. Currently, survey results are emailed to employees for manual review and sentiment analysis. The company wants to automate this process and retain survey data for the previous 12 months. Which solution offers the MOST scalable approach to meet these requirements?

- ☒ Send survey results to an Amazon API Gateway endpoint connected to an Amazon SQS queue. An AWS Lambda function polls the queue, uses Amazon Comprehend for sentiment analysis, and stores results in a DynamoDB table with a TTL of 365 days.

This solution is highly scalable, leverages serverless architecture, uses Amazon Comprehend for sentiment analysis, and ensures data retention for 12 months with DynamoDB's TTL feature. SQS handles the high volume of requests, and Lambda enables automatic processing without the need of managing servers.

- ☐ Send survey results to an API on an Amazon EC2 instance. The API stores results in a DynamoDB table, calls Amazon Comprehend for sentiment analysis, and saves the sentiment data in a second DynamoDB table with a TTL of 365 days.

While this approach utilizes DynamoDB and Comprehend, using an EC2 instance to handle the API creates a potential bottleneck and scalability issues, especially given the thousands of surveys received every hour. It also necessitates managing and scaling the EC2 instance, adding operational overhead.

- ☐ Write survey results to an Amazon S3 bucket. Use S3 Event Notifications to invoke an AWS Lambda function to read the data and call Amazon Rekognition for sentiment analysis. Store the sentiment analysis results in a second S3 bucket. Use S3 lifecycle policies on each bucket to expire objects after 365 days.

Amazon Rekognition is designed for image and video analysis, not text-based sentiment analysis. While using S3 and Lambda provides some scalability, Rekognition is the wrong tool for this problem and storing sentiment in a separate bucket adds unnecessary complexity.

- ☐ Send survey results to an Amazon API Gateway endpoint connected to an Amazon SQS queue. Configure the SQS queue to invoke an AWS Lambda function that calls Amazon Lex for sentiment analysis and saves the results to an Amazon DynamoDB table. Set the TTL for all records to 365 days in the future.

Amazon Lex is mainly used for building conversational interfaces, such as chatbots, and not for general sentiment analysis. Although the other components (API Gateway, SQS, Lambda, DynamoDB) offer scalability, Lex is not the appropriate service for this use case. Amazon Comprehend is better suited for Sentiment Analysis.

Overall Explanation:

The MOST scalable solution uses Amazon API Gateway, Amazon SQS, AWS Lambda, Amazon Comprehend, and Amazon DynamoDB. API Gateway provides a scalable entry point, SQS decouples the survey data ingestion from processing, Lambda allows for serverless execution of the sentiment analysis, Comprehend provides sentiment analysis based on text, and DynamoDB provides a highly scalable and flexible database with TTL for automatic data expiration.

[View Discussions](#)[Ask AI](#)

Question 10

Not Attempted

[Automation](#)

A company uses AWS Systems Manager for routine management and patching of Amazon EC2 instances. The EC2 instances are in an IP address type target group behind an Application Load Balancer (ALB). New security protocols require the company to remove EC2 instances from service during patching. During the next patching window, the company encounters errors. Which combination of solutions will resolve the errors?

- ☐ ☒ Change the target type of the target group from IP address type to instance type.
Changing the target type is unnecessary when existing tools can handle the issue more effectively.
- ☐ ☒ Continue to use the existing Systems Manager document without changes because it is already optimized to handle instances that are in an IP address type target group behind an ALB.
The existing document is not optimized for instances in an IP address type target group, so errors would persist.
- ☒ ☐ Implement the AWSEC2-PatchLoadBalancerInstance Systems Manager Automation document to manage the patching process.
This document automates the process of deregistering instances from the ALB during patching and re-registering them afterward, ensuring no traffic is routed to the instances while they are being patched. It directly addresses the issue and adheres to best practices.

- ✔ Use Systems Manager Maintenance Windows to automatically remove the instances from service to patch the instances.
Maintenance Windows allow you to define a schedule for patching and ensure instances are temporarily removed from service during the process.
- ✘ Configure Systems Manager State Manager to remove the instances from service and manage the patching schedule. Use ALB health checks to re-route traffic.
State Manager alone cannot deregister and re-register instances from the ALB target group.

Overall Explanation:

The errors occur because the company is trying to patch instances that are still actively serving traffic. `AWSEC2-PatchLoadBalancerInstance` Systems Manager Automation document helps to deregister and register the instances from ALB. Also, Systems Manager Maintenance Windows ensures instances are temporarily removed from service during the patching process. Therefore, implementing `AWSEC2-PatchLoadBalancerInstance` Systems Manager Automation document and Using Systems Manager Maintenance Windows will solve the issue.

[View Discussions](#)[Ask AI](#)

Question 11

Not Attempted

Serverless



A medical company needs to transform clinical trial data from multiple customers. The data resides in a relational database and needs to be extracted, transformed using complex rules, and then loaded into Amazon S3. Each customer's data must be encrypted with a customer-specific key before being stored in S3. Which solution meets these requirements with the LEAST amount of operational effort?

- ✘ Create one AWS Glue job for each customer. Attach a security configuration to each job that uses server-side encryption with Amazon S3 managed keys (SSE-S3) to encrypt the data.
SSE-S3 uses S3 managed keys, which does not meet the requirement of customer-specific keys. Also, server-side encryption would occur after the data is written to S3, not during processing.
- ✘ Create one Amazon EMR cluster for each customer. Attach a security configuration to each cluster that uses client-side encryption with a custom client-side root key (CSE-

Custom) to encrypt the data.

While CSE-Custom addresses the customer-specific key requirement, managing EMR clusters for each customer involves significant operational overhead. AWS Glue provides a more managed and serverless approach.

- ✔ Create one AWS Glue job for each customer. Attach a security configuration to each job that uses client-side encryption with AWS KMS managed keys (CSE-KMS) to encrypt the data.

This option leverages AWS Glue for a serverless ETL process, reducing operational effort. CSE-KMS allows for customer-specific KMS keys to encrypt the data before it is stored in S3, meeting all requirements.

- ✘ Create one Amazon EMR cluster for each customer. Attach a security configuration to each cluster that uses server-side encryption with AWS KMS keys (SSE-KMS) to encrypt the data.

While SSE-KMS allows for KMS keys, server-side encryption occurs after the data has already been written to S3 and managing EMR clusters for each customer increases operational complexity. The requirement is to encrypt 'where it is processed'.

Overall Explanation:

Option C is the most efficient solution. AWS Glue provides a serverless ETL environment that minimizes operational overhead. Using client-side encryption with AWS KMS (CSE-KMS) allows for encryption with customer-specific KMS keys *before* the data is stored in S3, fulfilling the data protection requirements and ensuring compliance with the need for customer-specific keys during processing. The other options either do not meet the customer-specific key requirement, or add significant operational management overhead with the need to manage and maintain separate EMR clusters for each customer.

[View Discussions](#)[Ask AI](#)

Question 12

Not Attempted

[Compute](#)

A company hosts a website analytics application on a single Amazon EC2 On-Demand Instance. The analytics application is highly resilient and is designed to run in stateless mode. The company notices that the application is showing signs of performance degradation during busy times and is presenting 5xx errors. The company needs to make the application scale

seamlessly and cost-effectively. Which solution will meet these requirements MOST cost-effectively?

- ☐ Create an Amazon Machine Image (AMI) of the web application. Use the AMI to launch a second EC2 On-Demand Instance. Use an Application Load Balancer to distribute the load across the two EC2 instances.
This solution provides some scaling but does not automatically adjust to changing demand, and it's not the most cost-effective.
- ☐ Create an Amazon Machine Image (AMI) of the web application. Use the AMI to launch a second EC2 On-Demand Instance. Use Amazon Route 53 weighted routing to distribute the load across the two EC2 instances.
Weighted routing is not ideal for seamless scaling because it requires manual intervention to adjust weights or add new instances, making it unsuitable for dynamic traffic changes.
- ☐ Create an AWS Lambda function to stop the EC2 instance and change the instance type. Create an Amazon CloudWatch alarm to invoke the Lambda function when CPU utilization is more than 75%.
This solution introduces downtime during instance type changes and doesn't provide seamless scalability. Additionally, stopping and resizing an instance is not a scaling solution, but rather a temporary fix.
- ☒ Create an Amazon Machine Image (AMI) of the web application. Apply the AMI to a launch template. Create an Auto Scaling group that includes the launch template. Configure the launch template to use a Spot Fleet. Attach an Application Load Balancer to the Auto Scaling group.
This solution provides automatic scaling based on demand using an Auto Scaling group, leverages cost-effective Spot Instances, and distributes traffic efficiently using an Application Load Balancer, making it the most cost-effective and scalable option.

Overall Explanation:

The most cost-effective solution is to use an Auto Scaling group with a launch template using Spot instances, combined with an Application Load Balancer. This dynamically adjusts the number of instances based on demand (scalability), reduces costs significantly compared to On-Demand Instances (cost-effectiveness), and ensures efficient traffic routing to healthy instances. Since the application is stateless, it can easily scale horizontally without data consistency issues. Option A does not scale automatically. Option B requires manual intervention. Option C introduces downtime.

[View Discussions](#)[Ask AI](#)

Question 13

Not Attempted

Storage



A company stores data in an Amazon S3 bucket with frequent access throughout the day. Strict data encryption requirements are in place, and the company currently uses AWS Key Management Service (AWS KMS) for encryption. To optimize costs without increasing KMS API calls, which solution is most suitable for encrypting new S3 objects?

- ☐ Use server-side encryption with Amazon S3 managed keys (SSE-S3).
SSE-S3 provides encryption at rest but doesn't involve KMS, therefore it doesn't address the cost optimization of KMS calls.
- ☒ Use an S3 Bucket Key for server-side encryption with AWS KMS keys (SSE-KMS) on the new objects.
S3 Bucket Keys reduce the cost of SSE-KMS by decreasing the number of requests to AWS KMS. Subsequent requests that take advantage of the bucket-level key do not result in AWS KMS API requests.
- ☐ Use client-side encryption with AWS KMS customer managed keys.
Client-side encryption involves encryption before the data is sent to S3. While KMS is used, it doesn't optimize S3's KMS request costs.
- ☐ Use server-side encryption with customer-provided keys (SSE-C) stored in AWS KMS.
SSE-C requires managing your own encryption keys and doesn't help optimize the costs associated with KMS requests from S3.

Overall Explanation:

S3 Bucket Keys are designed to reduce the cost of Amazon S3 server-side encryption with AWS Key Management Service (AWS KMS) keys (SSE-KMS). They decrease the request traffic from Amazon S3 to AWS KMS, reducing AWS KMS request costs.

[View Discussions](#)[Ask AI](#)

Question 14

Not Attempted

Networking



A solutions architect is designing a VPC with public and private subnets. The VPC and subnets use IPv4 CIDR blocks. There is one public subnet and one private subnet in each of three Availability Zones (AZs) for high availability. An internet gateway is used to provide internet access for the public subnets. The private subnets require access to the internet to allow Amazon EC2 instances to download software updates. What should the solutions architect do to enable Internet access for the private subnets?

- ☒ Create three NAT gateways, one for each public subnet in each AZ. Create a private route table for each AZ that forwards non-VPC traffic to the NAT gateway in its AZ.
This is the correct approach. NAT gateways in public subnets allow private subnets to initiate outbound traffic to the internet while preventing inbound traffic. Route tables direct non-VPC traffic from private subnets to the appropriate NAT gateway.
- ☐ Create three NAT instances, one for each private subnet in each AZ. Create a private route table for each AZ that forwards non-VPC traffic to the NAT instance in its AZ.
NAT instances are a legacy approach. They require manual management and are less scalable and resilient than NAT gateways.
- ☐ Create a second internet gateway on one of the private subnets. Update the route table for the private subnets that forward non-VPC traffic to the private internet gateway.
Internet gateways are attached to the VPC, not individual subnets, and are designed for public subnets. Attaching one to a private subnet is not a valid configuration.
- ☐ Create an egress-only internet gateway on one of the public subnets. Update the route table for the private subnets that forward non-VPC traffic to the egress-only Internet gateway.
Egress-only internet gateways are used for IPv6 traffic, not IPv4. Also, they do not allow inbound traffic.

Overall Explanation:

The most appropriate solution is to create NAT gateways in each public subnet (one per Availability Zone) and configure the route tables for the private subnets to route internet-bound traffic to the corresponding NAT gateway in their AZ. NAT gateways allow EC2 instances in private subnets to download software updates and access other resources on the Internet without being directly exposed to the internet. This design provides high availability by ensuring that each AZ has its own NAT gateway.

[View Discussions](#)[Ask AI](#)

Question 15

Not Attempted

[Compute](#)

A company is undergoing rapid expansion and needs to migrate its on-premises workloads to AWS due to scalability limitations of its current data center. The migration is time-sensitive, and the company aims to use a lift-and-shift strategy for its non-critical workloads. Which combination of actions will efficiently meet these requirements?

- ☐ Use the AWS Schema Conversion Tool (AWS SCT) to collect data about the VMs.
AWS SCT is primarily for database schema conversions and is not directly involved in migrating VMs using a lift-and-shift approach.
- ☒ Use AWS Application Migration Service. Install the AWS Replication Agent on the VMs.
AWS Application Migration Service (MGN) is designed for lift-and-shift migrations. Installing the Replication Agent enables the service to replicate the VMs to AWS.
- ☒ Complete the initial replication of the VMs. Launch test instances to perform acceptance tests on the VMs.
Testing the migrated instances is a crucial step to validate their functionality and ensure a smooth transition.
- ☒ Stop all operations on the VMs. Launch a cutover instance.
Stopping operations and launching the cutover instance represents the final switch from the on-premises environment to AWS.
- ☐ Use AWS App2Container (A2C) to collect data about the VMs.
AWS App2Container is used for containerizing applications. It's not suitable for a general lift-and-shift VM migration.
- ☐ Use AWS Database Migration Service (AWS DMS) to migrate the VMs.
AWS DMS is specifically designed for database migrations and is not used for migrating entire VMs.

Overall Explanation:

To efficiently migrate non-critical workloads to AWS using a lift-and-shift strategy, the correct steps are: using AWS Application Migration Service (MGN) with its Replication Agent (Option B), performing acceptance tests after the initial replication (Option C), and

executing the cutover by stopping on-premises operations and launching the cutover instance (Option D). This approach allows for a quick and relatively simple migration.

[View Discussions](#)[Ask AI](#)

Question 16

Not Attempted

Security



A company hosts an application in a private subnet. The company has integrated the application with Amazon Cognito using a user pool for authentication. The company now requires the application to securely store user documents in an Amazon S3 bucket. Which combination of steps will achieve secure integration between the application and Amazon S3?

- ☒ Create an Amazon Cognito identity pool to generate secure Amazon S3 access tokens for users upon successful login.

Amazon Cognito identity pools provide temporary AWS credentials, including access tokens, for authenticated users, enabling secure access to AWS services like S3.

- ☐ Use the existing Amazon Cognito user pool to generate Amazon S3 access tokens for users upon successful login.

User pools are for authentication (verifying user identity), while identity pools are for authorization (granting access to AWS resources). User pools cannot directly generate S3 access tokens.

- ☒ Create an Amazon S3 VPC endpoint in the same VPC where the company hosts the application.

An S3 VPC endpoint ensures that traffic between the VPC and S3 remains within the AWS network, enhancing security and removing the need for a NAT gateway for S3 access.

- ☐ Create a NAT gateway in the VPC where the company hosts the application. Assign a policy to the S3 bucket to deny any request that is not initiated from Amazon Cognito.

A NAT gateway allows instances in private subnets to connect to the internet, but doesn't inherently secure access to S3. While a bucket policy can restrict access, the identity pool provides better, role-based security.

- ☐ Attach a policy to the S3 bucket that allows access only from the users' IP addresses.

Restricting access based on user IP addresses is unreliable as IP addresses can change frequently, especially for mobile users.

Overall Explanation:

To securely integrate Amazon S3 with the application, an Amazon Cognito identity pool should be created to generate temporary AWS credentials for authenticated users after they successfully log in. Also, an Amazon S3 VPC endpoint should be created in the same VPC where the company hosts the application to ensure all traffic to S3 remains within the AWS network.

[View Discussions](#)[Ask AI](#)

Question 17

Not Attempted

[Compute](#)

A company runs a three-tier web application processing customer orders. The web tier uses Amazon EC2 instances behind an Application Load Balancer. The processing tier also uses EC2 instances. The company has decoupled the web and processing tiers using Amazon SQS. The storage layer utilizes Amazon DynamoDB. During peak times, users experience order processing delays. The EC2 instances in the processing tier run at 100% CPU usage, and the SQS queue fills up. These peak times are variable and unpredictable. What is the MOST cost-effective solution to improve the application's performance and prevent processing delays?

- ☐ Use scheduled scaling for Amazon EC2 Auto Scaling to scale out the processing tier instances during peak usage times, using CPU Utilization as the scaling metric.
Scheduled scaling isn't suitable because peak times are unpredictable. CPU utilization alone might not accurately reflect queue backlog.
- ☐ Implement Amazon ElastiCache for Redis in front of DynamoDB. Use target utilization as a metric to determine when to scale ElastiCache instances.
Caching DynamoDB reads might help, but the bottleneck is in the processing tier, not the database reads. Scaling ElastiCache here does not address the root cause.
- ☐ Add an Amazon CloudFront distribution to cache web tier responses. Use HTTP latency as a metric to determine when to scale CloudFront.
CloudFront improves web tier performance, but the bottleneck is in the processing tier, making this solution ineffective.
- ☒ Use an Amazon EC2 Auto Scaling target tracking policy to scale out the processing tier instances based on the SQS queue's `ApproximateNumberOfMessages` attribute.
This is the MOST appropriate solution. Target tracking based on `ApproximateNumberOfMessages` in the SQS queue directly addresses the backlog in the processing tier. It dynamically scales the

processing instances based on the queue length, handling unpredictable peak times effectively.

Overall Explanation:

The bottleneck lies within the processing tier. Options B and C address other parts of the architecture and will not solve the main issue. Option A uses scheduled scaling, which is not ideal as peak times are unpredictable. Option D is the best solution because it automatically scales the processing tier based on the number of messages in the SQS queue (ApproximateNumberOfMessages), thus directly responding to processing load and preventing delays. Using target tracking ensures the processing tier scales dynamically and cost-effectively based on queue length. CPU utilization may not be an accurate metric. There could be just one task but it takes quite a while to process, in which case scaling based on CPU is unnecessary.

[View Discussions](#)[Ask AI](#)

Question 18

Not Attempted

[Compute](#)

A company's production environment uses Amazon EC2 On-Demand Instances that run continuously from Monday to Saturday. The instances are required for only 12 hours on Sunday and cannot withstand any interruptions. The company aims to optimize the costs of its production environment. Which solution is the MOST cost-effective while meeting these requirements?

- ☒ Purchase Scheduled Reserved Instances for the EC2 instances that run for only 12 hours on Sunday. Purchase Standard Reserved Instances for the EC2 instances that run constantly between Monday and Saturday.

Scheduled Reserved Instances are ideal for predictable, recurring schedules like the 12 hours on Sunday and provide guaranteed capacity. Standard Reserved Instances offer cost savings for the instances running continuously during the week.

- ☐ Purchase Convertible Reserved Instances for the EC2 instances that run for only 12 hours on Sunday. Purchase Standard Reserved Instances for the EC2 instances that run constantly between Monday and Saturday.

Convertible Reserved Instances provide flexibility to change instance attributes but are more expensive than Scheduled Reserved Instances, making them less cost-effective for the Sunday workload.

- ⊗ Use Spot Instances for the EC2 instances that run for only 12 hours on Sunday. Purchase Standard Reserved Instances for the EC2 instances that run constantly between Monday and Saturday.

Spot Instances can be interrupted, violating the requirement that instances cannot tolerate interruptions. They are therefore not suitable for the Sunday workload.

- ⊗ Use Spot Instances for the EC2 instances that run for only 12 hours on Sunday. Purchase Convertible Reserved Instances for the EC2 instances that run constantly between Monday and Saturday.

Spot Instances are unsuitable due to potential interruptions. While Convertible Reserved Instances are flexible, they are not necessary for the continuous weekday workload where instance attributes are unlikely to change, and standard RI is cheaper.

Overall Explanation:

The most cost-effective solution is to use Scheduled Reserved Instances for the 12 hours on Sunday, as they are designed for predictable recurring schedules and guarantee capacity. Standard Reserved Instances are the most cost-effective option for the continuously running instances from Monday to Saturday. Spot Instances are not suitable due to the possibility of interruptions. Convertible Reserved Instances are more expensive and provide unnecessary flexibility for the Sunday workload, as compared to Scheduled Reserved Instances, and more expensive than standard RI for instances that always run.

 View Discussions

 Ask AI

Question 19

Not Attempted

Serverless



A digital image processing company wants to migrate its on-premises monolithic application to the AWS Cloud. The company processes thousands of images and generates large files as part of the processing workflow. The company needs a solution to manage the growing number of image processing jobs. The solution must also reduce the manual tasks in the image processing workflow. The company does not want to manage the underlying infrastructure of the solution. Which solution will meet these requirements with the LEAST operational overhead?

- ⊗ Use Amazon Elastic Container Service (Amazon ECS) with Amazon EC2 Spot Instances to process the images. Configure Amazon Simple Queue Service (Amazon SQS) to

orchestrate the workflow. Store the processed files in Amazon Elastic File System (Amazon EFS).

Using ECS with Spot Instances and SQS involves more manual configuration and infrastructure management. EFS can also be more expensive than S3 for storing large files.

- ✔ Use AWS Batch jobs to process the images. Use AWS Step Functions to orchestrate the workflow. Store the processed files in an Amazon S3 bucket.

AWS Batch offers a fully managed batch processing service, automatically scaling resources based on job requirements. AWS Step Functions provides visual workflow orchestration, reducing manual intervention. Amazon S3 offers scalable and cost-effective storage for large files.

- ✘ Use AWS Lambda functions and Amazon EC2 Spot Instances to process the images. Store the processed files in Amazon FSx.

Lambda functions might have limitations on processing time for large image processing tasks. FSx is a managed Windows or Lustre file system, not an ideal storage choice compared to S3 for this use case. Combining with Spot Instances introduces potential for interruptions.

- ✘ Deploy a group of Amazon EC2 instances to process the images. Use AWS Step Functions to orchestrate the workflow. Store the processed files in an Amazon Elastic Block Store (Amazon EBS) volume.

Managing a group of EC2 instances involves significant operational overhead, including patching, scaling, and monitoring. EBS volumes are tied to specific EC2 instances, making scalability and management more complex.

Overall Explanation:

AWS Batch combined with Step Functions and S3 offers the least operational overhead by providing a fully managed batch processing environment, robust workflow orchestration, and scalable storage. AWS Batch eliminates the need to manage underlying infrastructure, allowing the company to focus on image processing jobs rather than server management. Step Functions further reduces manual tasks through automated workflows. S3 provides scalable and cost effective storage. Using Spot Instances (A & C) are cost-effective but are subject to interruptions, increasing manual tasks. EC2 instances require managing the underlying infrastructure (D). Amazon FSx (C) is not an ideal choice compared to s3 (A).

 View Discussions

 Ask AI

A company's image-hosting website allows users worldwide to upload, view, and download images from mobile devices. The static website is currently hosted in an Amazon S3 bucket. Due to the website's increasing popularity, performance has decreased, with users reporting latency issues during image uploads and downloads. The company needs to improve website performance with the LEAST implementation effort. Which solution meets these requirements?

- ☒ **Configure an Amazon CloudFront distribution for the S3 bucket to improve download performance. Enable S3 Transfer Acceleration to improve upload performance.**
This solution leverages CloudFront for faster content delivery globally and S3 Transfer Acceleration for optimized uploads. It is a cost-effective and relatively simple solution to implement.
- ☐ **Configure Amazon EC2 instances of the right sizes in multiple AWS Regions. Migrate the application to the EC2 instances. Use an Application Load Balancer to distribute website traffic equally among the EC2 instances. Configure AWS Global Accelerator to address global demand with low latency.**
This solution involves a complex migration to EC2 instances and is significantly more effort than using CloudFront and S3 Transfer Acceleration.
- ☐ **Configure an Amazon CloudFront distribution that uses the S3 bucket as an origin to improve download performance. Configure the application to use CloudFront to upload images to improve upload performance. Create S3 buckets in multiple AWS Regions. Configure replication rules for the buckets to replicate users' data based on the users' location. Redirect downloads to the S3 bucket that is closest to each user's location.**
While this solution addresses latency, it involves unnecessary complexity with multi-region S3 buckets and replication rules. Using S3 Transfer Acceleration is a simpler alternative for upload performance.
- ☐ **Configure AWS Global Accelerator for the S3 bucket to improve network performance. Create an endpoint for the application to use Global Accelerator instead of the S3 bucket.**
Global Accelerator doesn't directly integrate with S3 like CloudFront does, and requires application-level changes to direct traffic through the accelerator. S3 Transfer Acceleration in conjunction with CloudFront is better suited for this purpose.

Overall Explanation:

The most efficient solution is to use Amazon CloudFront for improved download speeds and S3 Transfer Acceleration for accelerated uploads. CloudFront, acting as a CDN, caches the content closer to users. S3 Transfer Acceleration utilizes CloudFront's edge locations to

optimize the upload path to the S3 bucket. This approach minimizes implementation effort while addressing the identified performance issues.

[View Discussions](#)[Ask AI](#)

Question 21

Not Attempted

Networking



A company hosts an application within a private subnet of a VPC. The VPC has a NAT gateway and an internet gateway. The application needs to store objects in Amazon S3. The company's security policy strictly prohibits application traffic from traversing the public internet. What is the MOST cost-effective solution to meet these requirements?

- ☐ Configure an S3 interface endpoint. Create a security group that allows outbound traffic to Amazon S3.

An interface endpoint uses AWS PrivateLink, providing private connectivity to S3. However, it's generally more expensive than a gateway endpoint.

- ☒ Configure an S3 gateway endpoint. Update the VPC route table to use the endpoint.

A gateway endpoint provides cost-effective and reliable connectivity to S3 without traversing the internet. Updating the route table ensures traffic destined for S3 uses the endpoint.

- ☐ Configure an S3 bucket policy to allow traffic from the Elastic IP address that is assigned to the NAT gateway.

This approach still requires traffic to traverse the NAT gateway and the internet, violating the security policy.

- ☐ Create a second NAT gateway in the same subnet where the legacy application is deployed. Update the VPC route table to use the second NAT gateway.

Creating a second NAT gateway does not prevent traffic from traversing the internet and is not cost-effective. NAT gateways also introduce additional overhead.

Overall Explanation:

An S3 gateway endpoint provides the most cost-effective solution by enabling direct connectivity to S3 from within the VPC without using the internet. It's cheaper than an interface endpoint and satisfies the requirement that traffic must not travel across the

internet. The VPC route table update ensures that S3-bound traffic is directed to the gateway endpoint. Therefore the correct answer is B.

[View Discussions](#)[Ask AI](#)

Question 22

Not Attempted

Security



A company has an application running on an Amazon Elastic Kubernetes Service (Amazon EKS) cluster on Amazon EC2 instances. The application's UI uses Amazon DynamoDB, and its data services utilize Amazon S3 as part of the deployment. The company requires that EKS Pods for the UI can access only Amazon DynamoDB, and EKS Pods for data services can access only Amazon S3. The company uses AWS Identity and Access Management (IAM). Which solution meets these requirements?

- ☐ Create separate IAM policies for Amazon S3 and DynamoDB access with the required permissions. Attach both IAM policies to the EC2 instance profile. Use role-based access control (RBAC) to control access to Amazon S3 or DynamoDB for the respective EKS Pods.

Attaching both IAM policies to the EC2 instance profile does not provide granular control at the Pod level. RBAC within Kubernetes is for authorization within the cluster and doesn't directly map to AWS resource access.

- ☐ Create separate IAM policies for Amazon S3 and DynamoDB access with the required permissions. Attach the Amazon S3 IAM policy directly to the EKS Pods for the data services and the DynamoDB policy to the EKS Pods for the UI.

Directly attaching IAM policies to Pods is not a supported or recommended approach. IAM roles are assumed via service accounts.

- ☒ Create separate Kubernetes service accounts for the UI and data services to assume an IAM role. Attach the AmazonS3FullAccess policy to the data services account and the AmazonDynamoDBFullAccess policy to the UI service account.

This approach uses Kubernetes service accounts and IAM roles, but grants overly permissive access (FullAccess). While it technically works, it violates the principle of least privilege.

- ☐ Create separate Kubernetes service accounts for the UI and data services to assume an IAM role. Use IAM Role for Service Accounts (IRSA) to provide access to the EKS Pods for the UI to Amazon S3 and the EKS Pods for the data services to DynamoDB.

This option incorrectly assigns the services. The UI Pods need access to DynamoDB, and the data services Pods need access to S3, not the other way around.

Overall Explanation:

The best solution involves using Kubernetes service accounts and IAM Roles for Service Accounts (IRSA) to grant specific permissions to the Pods. While option C uses FullAccess policies, it aligns most closely with the correct approach of associating service accounts with IAM roles, ensuring the UI Pods access DynamoDB and data services Pods access S3. Options A and B are incorrect because they either lack granular control or suggest unsupported methods. Option D is incorrect because it swaps the required permissions.

[View Discussions](#)[Ask AI](#)

Question 23

Not Attempted

Security



A company with a globally distributed development team needs to grant them secure access to the company's AWS resources while adhering to security policies. The company utilizes an on-premises Active Directory for internal authentication and AWS Organizations to manage multiple AWS accounts across different projects. The desired solution should integrate with the existing infrastructure to provide centralized identity management and access control with the LEAST operational overhead. Which solution best meets these requirements?

- ☐ Set up AWS Directory Service to create an AWS managed Microsoft Active Directory on AWS. Establish a trust relationship with the on-premises Active Directory. Use IAM roles that are assigned to Active Directory groups to access AWS resources within the company's AWS accounts.

This option introduces higher overhead by requiring the creation and maintenance of a separate Active Directory instance in AWS.

- ☐ Create an IAM user for each developer. Manually manage permissions for each IAM user based on each user's involvement with each project. Enforce multi-factor authentication (MFA) as an additional layer of security.

This option introduces significant scalability issues due to the manual creation and management of IAM users, especially for a globally distributed team.

- ☒ Use AD Connector in AWS Directory Service to connect to the on-premises Active Directory. Integrate AD Connector with AWS IAM Identity Center. Configure

permissions sets to give each AD group access to specific AWS accounts and resources. AD Connector seamlessly connects AWS to the on-premises Active Directory without synchronizing or replicating the directory. IAM Identity Center allows centralized access management across AWS accounts in AWS Organizations. Permissions sets can map Active Directory groups to specific AWS accounts and resources, simplifying access control and enhancing security. Once set up, it requires minimal ongoing management. User provisioning and deprovisioning are handled through the existing Active Directory processes.

- ⊗ Use Amazon Cognito to deploy an identity federation solution. Integrate the identity federation solution with the on-premises Active Directory. Use Amazon Cognito to provide access tokens for developers to access AWS accounts and resources.

Amazon Cognito is better suited for application-level identity management (like customer-facing apps), not for internal teams working across multiple AWS accounts. Also this introduces an unnecessary federation layer.

Overall Explanation:

The best solution is to use AD Connector in AWS Directory Service to connect to the on-premises Active Directory and integrate it with AWS IAM Identity Center. Configuring permissions sets to give each AD group access to specific AWS accounts and resources provides centralized identity management and access control with the least operational overhead. Options A and B require more manual management or setup. Option D is more suitable for external identities.

 View Discussions

 Ask AI

Question 24

Not Attempted

Security



A company is developing an application in the AWS Cloud. The application's HTTP API contains critical information that is published in Amazon API Gateway. The critical information must be accessible from only a limited set of trusted IP addresses that belong to the company's internal network. Which solution will meet these requirements?

- ⊗ Set up an API Gateway private integration to restrict access to a predefined set of IP addresses.

A private integration results in a private API accessible only within a VPC, not based on IP address filtering.

- ☒ Create a resource policy for the API that denies access to any IP address that is not specifically allowed.

API Gateway resource policies can be used to restrict access based on the source IP address of the request. This allows you to explicitly allow only the company's internal network IP addresses.

- ☐ Directly deploy the API in a private subnet. Create a network ACL. Set up rules to allow the traffic from specific IP addresses.

While deploying the API in a private subnet isolates it, network ACLs act as a firewall for the entire subnet and can be used to restrict access based on IP addresses, this is less direct than using an API Gateway resource policy.

- ☐ Modify the security group that is attached to API Gateway to allow inbound traffic from only the trusted IP addresses.

Security groups cannot be directly attached to API Gateway.

Overall Explanation:

The most direct and effective solution is to use an API Gateway resource policy to explicitly allow access only from the trusted IP addresses. Resource policies provide fine-grained control over API access at the API Gateway level. Options A and C involve deploying the API within a private network, while limiting access using the network configurations. Option D isn't possible as security groups don't attach to API Gateway.

 View Discussions

 Ask AI

Question 25

Not Attempted

Storage



A company is migrating its on-premises data center to AWS. The data center hosts an SFTP server with 200 GB of data stored on an NFS-based file system. The requirement is to migrate this data to an Amazon EC2 instance leveraging an Amazon Elastic File System (Amazon EFS) file system. What combination of actions should a solutions architect implement to automate this data transfer? (Choose two.)

- ☐ Launch the EC2 instance into the same Availability Zone as the EFS file system.

Launching the EC2 instance in the same AZ as EFS ensures low-latency access to the EFS file system and avoids cross-AZ data transfer costs.

- ☒ Install an AWS DataSync agent in the on-premises data center.
AWS DataSync agent is required in the on-premises environment to facilitate the data transfer from the on-premises SFTP server to AWS.
- ☐ Create a secondary Amazon Elastic Block Store (Amazon EBS) volume on the EC2 instance for the data.
A secondary EBS volume is not necessary, the data will be moved directly to EFS.
- ☐ Manually use an operating system copy command to push the data to the EC2 instance.
Manual copying is not an automated solution, therefore is not correct.
- ☒ Use AWS DataSync to create a location configuration for the on-premises SFTP server.
AWS DataSync can be used to create a location configuration for the on-premises SFTP server, and then to move data to the EFS file system associated with the EC2 instance.

Overall Explanation:

To automate the data transfer from the on-premises SFTP server to an EC2 instance using EFS, you need to install an AWS DataSync agent in the on-premises data center and configure AWS DataSync to create a suitable location configuration for the SFTP server. AWS DataSync automates the data transfer process. Launching the EC2 instance in the same AZ is best practice for the application but not directly required for the automation of the data migration.

[View Discussions](#)[Ask AI](#)

Question 26

Not Attempted

Storage



A company has an AWS Glue extract, transform, and load (ETL) job that runs every day at the same time. The job processes XML data that is in an Amazon S3 bucket. New data is added to the S3 bucket every day. A solutions architect notices that AWS Glue is processing all the data during each run. What should the solutions architect do to prevent AWS Glue from reprocessing old data?

- ☒ Edit the job to use job bookmarks.
Job bookmarks in Glue allow you to track the last-processed data in a job. By enabling job bookmarks, Glue keeps track of the processed data and automatically resumes processing from

where it left off in subsequent job runs. This prevents reprocessing of data already processed.

⊗ Edit the job to delete data after the data is processed.

Deleting data after processing results in the permanent removal of the data from the S3 bucket, making it unavailable for future job runs or analysis. This is undesirable if the data needs to be retained.

⊗ Edit the job by setting the NumberOfWorkers field to 1.

Setting the NumberOfWorkers field to 1 affects the parallelism of the job but does not address the issue of reprocessing old data. It doesn't provide a mechanism to track processed data or skip already processed data.

⊗ Use a FindMatches machine learning (ML) transform.

The FindMatches transform is used for identifying and matching duplicate or similar records in a dataset. While it can be used in data processing pipelines, it doesn't address the specific requirement of avoiding reprocessing old data.

Overall Explanation:

Job bookmarks are the correct solution. They enable AWS Glue to track the data that has been processed in previous runs, ensuring that only new data is processed in subsequent runs. This avoids reprocessing, saves time, and reduces costs. Options B, C, and D do not address the root issue of preventing data reprocessing.

 View Discussions

 Ask AI

Question 27

Not Attempted

Security



A solutions architect must design a highly available infrastructure for a website. The website is powered by Windows web servers that run on Amazon EC2 instances. The solutions architect must implement a solution that can mitigate a large-scale DDoS attack that originates from thousands of IP addresses. Downtime is not acceptable for the website. Which actions should the solutions architect take to protect the website from such an attack? (Choose two.)

✓ Use AWS Shield Advanced to stop the DDoS attack.

AWS Shield Advanced provides enhanced DDoS protection for AWS resources, including EC2 instances. It offers real-time threat intelligence and automatic mitigation techniques to protect against large-scale attacks.

⊗ Configure Amazon GuardDuty to automatically block the attackers.

While Amazon GuardDuty can detect potential malicious activity, it is not specifically designed to mitigate DDoS attacks. It focuses on threat detection within your AWS environment.

✓ Configure the website to use Amazon CloudFront for both static and dynamic content.

Amazon CloudFront is a content delivery network (CDN) that distributes content across multiple edge locations. This helps absorb the impact of a DDoS attack by spreading the traffic across the network, reducing the load on the origin servers and improving availability.

⊗ Use an AWS Lambda function to automatically add attacker IP addresses to VPC network ACLs.

Network ACLs have limitations in terms of the number of rules and are not designed to handle high-volume DDoS traffic efficiently. Using a Lambda function to update ACLs would likely be ineffective and could introduce performance bottlenecks.

⊗ Use EC2 Spot Instances in an Auto Scaling group with a target tracking scaling policy that is set to 80% CPU utilization.

While Auto Scaling can help maintain availability during periods of high traffic, Spot Instances are not a reliable solution for DDoS mitigation, as they can be interrupted. This option addresses scaling, not DDoS protection.

Overall Explanation:

The best solution is to use AWS Shield Advanced and Amazon CloudFront. AWS Shield Advanced is designed to protect against DDoS attacks. CloudFront distributes your content across multiple edge locations, further mitigating the impact of the DDoS attack on your origin servers, ensuring high availability and low latency for your users.

 [View Discussions](#)

 [Ask AI](#)

Question 28

Not Attempted

Serverless



A company is preparing to deploy a new serverless workload. A solutions architect must use the principle of least privilege to configure permissions that will be used to run an AWS Lambda function. An Amazon EventBridge (Amazon CloudWatch Events) rule will invoke the function. Which solution meets these requirements?

- ☐ Add an execution role to the function with `lambda:InvokeFunction` as the action and `*` as the principal.

This option grants permission to all principals, violating the principle of least privilege.

- ☐ Add an execution role to the function with `lambda:InvokeFunction` as the action and `Service: lambda.amazonaws.com` as the principal.

This option grants permission to the Lambda service itself, not EventBridge, which is required to invoke the function.

- ☐ Add a resource-based policy to the function with `lambda:*` as the action and `Service: events.amazonaws.com` as the principal.

This option grants EventBridge excessive permissions on the Lambda function. The principle of least privilege mandates granting only the necessary permissions.

- ☒ Add a resource-based policy to the function with `lambda:InvokeFunction` as the action and `Service: events.amazonaws.com` as the principal.

This option correctly grants EventBridge the permission to invoke the Lambda function, adhering to the principle of least privilege.

Overall Explanation:

The correct solution is to add a resource-based policy to the Lambda function that allows the `events.amazonaws.com` service to invoke it using the `lambda:InvokeFunction` action. This adheres to the principle of least privilege by granting only the necessary permission for EventBridge to trigger the Lambda function. Options A, B, and C grant broader or incorrect permissions, violating this principle.

 [View Discussions](#)

 [Ask AI](#)

Question 29

Not Attempted

Security



A company is preparing to store confidential data in Amazon S3. For compliance reasons, the data must be encrypted at rest. Encryption key usage must be logged for auditing purposes. Keys must be rotated every year. Which solution meets these requirements and is the MOST operationally efficient?

- ☐ Server-side encryption with customer-provided keys (SSE-C)

SSE-C requires customers to provide their own encryption keys, but it does not provide key rotation or built-in logging of key usage.

⊗ Server-side encryption with Amazon S3 managed keys (SSE-S3)

SSE-S3 uses Amazon S3 managed keys for encryption, which simplifies key management but does not provide key rotation or detailed key usage logging. While SSE-S3 logging is now available in CloudTrail, it still lacks the ability to meet the yearly rotation requirement operationally efficiently.

⊗ Server-side encryption with AWS KMS keys (SSE-KMS) with manual rotation

SSE-KMS uses AWS KMS keys but requires manual rotation, which is less operationally efficient than the automatic key rotation.

✓ Server-side encryption with AWS KMS keys (SSE-KMS) with automatic rotation

SSE-KMS with automatic key rotation provides a secure and efficient way to encrypt data at rest in S3. It uses KMS to manage the encryption keys securely. With SSE-KMS, encryption keys can be automatically rotated using KMS key rotation feature. SSE-KMS provides built-in audit logging for encryption key usage through CloudTrail.

Overall Explanation:

The most operationally efficient solution is SSE-KMS with automatic key rotation. It satisfies the encryption at rest, key rotation, and key usage logging requirements with minimal operational overhead. SSE-C and SSE-S3 do not support automatic key rotation or built-in detailed logging. Manual rotation with SSE-KMS is less efficient.

 View Discussions

 Ask AI

Question 30

Not Attempted

Databases



A bicycle sharing company is developing a multi-tier architecture to track the location of its bicycles during peak operating hours. The company wants to use these data points in its existing analytics platform. A solutions architect must determine the most viable multi-tier option to support this architecture. The data points must be accessible from a REST API. Which action meets these requirements for storing and retrieving location data?

⊗ Use Amazon Athena with Amazon S3.

Amazon Athena is a query service that makes it easy to analyze data in Amazon S3 using standard SQL. While Athena can be used for analytics, it doesn't inherently provide a REST API for real-time data ingestion or processing. It is more suitable for querying data at rest.

⊗ Use Amazon API Gateway with AWS Lambda.

While API Gateway can create a REST API and Lambda can process data, Lambda alone may not be suitable for high-volume, continuous ingestion of location data. It lacks built-in capabilities for real-time data streaming and analysis, and would require additional services for storage and processing large amounts of data.

⊗ Use Amazon QuickSight with Amazon Redshift.

Amazon QuickSight is a business intelligence service for creating visualizations and dashboards. Amazon Redshift is a data warehouse. While this combination is useful for analyzing historical data, it does not provide real-time data ingestion or a REST API for accessing the location data.

✔ Use Amazon API Gateway with Amazon Kinesis Data Analytics.

This is the most suitable option. Amazon API Gateway provides a managed solution for exposing a REST API. Amazon Kinesis Data Analytics can handle real-time data streams, process and analyze the location data, and integrate with the company's existing analytics platform.

Overall Explanation:

Amazon API Gateway provides the required REST API endpoint. Amazon Kinesis Data Analytics is designed for real-time data streams such as location data. It can process, analyze, and store the location data efficiently, while enabling integration with the company's existing analytics platform.

 View Discussions

 Ask AI

Question 31

Not Attempted

Databases



A company has an automobile sales website that stores its listings in a database on Amazon RDS. When an automobile is sold, the listing needs to be removed from the website, and the data must be sent to multiple target systems. Which design should a solutions architect recommend to ensure scalability, reliability, and decoupling of the systems?

⊗ Create an AWS Lambda function triggered when the database on Amazon RDS is updated to send the information to an Amazon Simple Queue Service (Amazon SQS) queue for the targets to consume.

This option only utilizes a single SQS queue, making it difficult to reliably deliver the data to multiple target systems, as once a message is consumed, it is removed from the queue.

- ⊗ Create an AWS Lambda function triggered when the database on Amazon RDS is updated to send the information to an Amazon Simple Queue Service (Amazon SQS) FIFO queue for the targets to consume.

While FIFO queues maintain message order, this option still lacks a mechanism to reliably deliver the data to multiple independent target systems. FIFO is not required in this scenario.

- ⊗ Subscribe to an RDS event notification and send an Amazon Simple Queue Service (Amazon SQS) queue fanned out to multiple Amazon Simple Notification Service (Amazon SNS) topics. Use AWS Lambda functions to update the targets.

This option reverses the typical fan-out pattern, sending from a single SQS queue to multiple SNS topics. SQS has a many-to-one relationship not one-to-many. This is less efficient and doesn't effectively decouple the target systems.

- ✔ Subscribe to an RDS event notification and send an Amazon Simple Notification Service (Amazon SNS) topic fanned out to multiple Amazon Simple Queue Service (Amazon SQS) queues. Use AWS Lambda functions to update the targets.

This is the recommended approach. SNS fan-out allows sending data to multiple SQS queues, ensuring each target system receives the data. Lambda functions can then process messages from each SQS queue to update the respective targets, providing scalability and decoupling. RDS event notifications are triggered when an update happens on the DB.

Overall Explanation:

The optimal design is to subscribe to RDS event notifications and send the data to an Amazon SNS topic. The SNS topic is then fanned out to multiple Amazon SQS queues, each dedicated to a specific target system. AWS Lambda functions are triggered by these SQS queues to process the data and update the target systems. This approach provides scalability, reliability, and decoupling by ensuring that each target system receives the data independently and asynchronously. Options A, B and C do not provide a fan-out mechanism to send data to multiple target systems reliably.

 View Discussions

 Ask AI

Question 32

Not Attempted

Storage



A company needs to store data in Amazon S3 and must prevent the data from being changed. The company wants new objects that are uploaded to Amazon S3 to remain unchangeable for a nonspecific amount of time until the company decides to modify the objects. Only specific

users in the company's AWS account can have the ability to delete the objects. Which solution meets these requirements?

- ⊗ Create an S3 Glacier vault. Apply a write-once, read-many (WORM) vault lock policy to the objects.

Incorrect. S3 Glacier vault lock policies, while enforcing WORM, do not readily allow for deletion by specific users before the lock expires.

- ⊗ Create an S3 bucket with S3 Object Lock enabled. Enable versioning. Set a retention period of 100 years. Use governance mode as the S3 bucket's default retention mode for new objects.

Incorrect. Setting a retention period of 100 years contradicts the requirement for a nonspecific amount of time. The question requires the objects to remain unchangeable until the company decides to modify them, and a fixed retention period doesn't fulfill this need.

- ⊗ Create an S3 bucket. Use AWS CloudTrail to track any S3 API events that modify the objects. Upon notification, restore the modified objects from any backup versions that the company has.

Incorrect. This option does not prevent data from being changed initially but relies on detection and restoration from backups. This does not meet the requirement to prevent the data from being changed.

- ✔ Create an S3 bucket with S3 Object Lock enabled. Enable versioning. Add a legal hold to the objects. Add the `s3:PutObjectLegalHold` permission to the IAM policies of users who need to delete the objects.

Correct. S3 Object Lock with a legal hold allows for indefinite retention until the hold is removed. Versioning ensures that previous versions are retained. Granting specific users the `s3:PutObjectLegalHold` permission enables them to remove the legal hold and subsequently delete the object.

Overall Explanation:

The best solution is to use S3 Object Lock with a legal hold. A legal hold prevents an object version from being overwritten or deleted and remains in effect until removed. Enabling versioning ensures that all versions of the object are preserved. By adding the `s3:PutObjectLegalHold` permission to the IAM policies of specific users, these users can remove the legal hold when the company decides to modify the objects, thus enabling deletion. Options A, B, and C do not fulfill all requirements of the question.

[View Discussions](#)[Ask AI](#)

Question 33

Not Attempted

Security



A company's application runs on Amazon EC2 instances and uses an Amazon Aurora database. Currently, the EC2 instances connect to the database using usernames and passwords stored locally in a file. To minimize operational overhead related to credential management, what should a solutions architect recommend?



Use AWS Secrets Manager. Turn on automatic rotation.

AWS Secrets Manager provides a secure and centralized service for managing secrets like database credentials and supports automatic rotation, minimizing manual intervention and operational overhead.



Use AWS Systems Manager Parameter Store. Turn on automatic rotation.

While Parameter Store can store secrets, it does not offer automatic rotation capabilities without custom scripting, increasing operational overhead.



Create an Amazon S3 bucket to store objects that are encrypted with an AWS Key Management Service (AWS KMS) encryption key. Migrate the credential file to the S3 bucket. Point the application to the S3 bucket.

Storing credentials in S3, even encrypted, is less secure and requires more complex implementation than using Secrets Manager, adding to operational overhead.



Create an encrypted Amazon Elastic Block Store (Amazon EBS) volume for each EC2 instance. Attach the new EBS volume to each EC2 instance. Migrate the credential file to the new EBS volume. Point the application to the new EBS volume.

Using encrypted EBS volumes adds operational overhead for managing volumes individually and doesn't provide automatic rotation of credentials.

Overall Explanation:

AWS Secrets Manager with automatic rotation is the recommended solution. It centrally manages credentials, encrypts them with KMS, and automatically rotates them. Parameter Store does not offer auto-rotation. S3 and EBS solutions are more complex and less secure than using Secrets Manager. Therefore, using AWS Secrets Manager minimizes the operational overhead of credential management.

[View Discussions](#)[Ask AI](#)

Question 34

Not Attempted

Serverless



A social media company hosts its website on Amazon EC2 instances, allowing users to upload images. The website resizes images to a standard size during upload requests and stores them in Amazon S3. Users are reporting slow upload times. The company wants to reduce coupling and improve website performance with an operationally efficient solution. Which combination of actions should a solutions architect implement?

- ☐ ☒ **Configure the application to upload images to S3 Glacier.**
S3 Glacier is for archival and infrequent access; it's not suitable for image uploads in this scenario and would worsen performance.
- ☐ ☒ **Configure the web server to upload the original images to Amazon S3.**
While offloading the storage to S3 is good, the web server still handles the upload and resizing, maintaining the coupling and performance bottleneck.
- ☒ ☒ **Configure the application to upload images directly from each user's browser to Amazon S3 through the use of a presigned URL.**
Using presigned URLs allows users to upload directly to S3, bypassing the web server and reducing its load. This reduces coupling and improves upload speed.
- ☒ ☒ **Configure S3 Event Notifications to invoke an AWS Lambda function when an image is uploaded. Use the function to resize the image.**
S3 Event Notifications triggering a Lambda function for resizing decouples the resizing process from the upload request, improving performance by processing images asynchronously.
- ☐ ☒ **Create an Amazon EventBridge (Amazon CloudWatch Events) rule that invokes an AWS Lambda function on a schedule to resize uploaded images.**
Scheduled resizing introduces unnecessary delay and is not triggered by the upload event, making it less efficient than using S3 event notifications.

Overall Explanation:

The optimal solution involves direct uploads to S3 using pre-signed URLs (option C) to reduce the load on the EC2 instances and asynchronous image resizing with S3 event notifications and Lambda (option D). This approach reduces coupling by separating the

upload and resizing processes, leading to improved website performance and operational efficiency.

[View Discussions](#)[Ask AI](#)

Question 35

Not Attempted

[Compute](#)

A company recently migrated a message processing system to AWS. The system receives messages into an ActiveMQ queue running on an Amazon EC2 instance. Messages are processed by a consumer application running on Amazon EC2. The consumer application processes the messages and writes results to a MySQL database running on Amazon EC2. The company wants this application to be highly available with low operational complexity. Which architecture offers the HIGHEST availability?

- ☐ Add a second ActiveMQ server to another Availability Zone. Add an additional consumer EC2 instance in another Availability Zone. Replicate the MySQL database to another Availability Zone.

This option provides some redundancy, but it lacks automated failover and scaling, making it less resilient and more complex to manage.

- ☐ Use Amazon MQ with active/standby brokers configured across two Availability Zones. Add an additional consumer EC2 instance in another Availability Zone. Replicate the MySQL database to another Availability Zone.

This improves the availability of the message queue, but the consumer and database still rely on manual intervention for failover. It's also more operationally complex than using a managed database service.

- ☐ Use Amazon MQ with active/standby brokers configured across two Availability Zones. Add an additional consumer EC2 instance in another Availability Zone. Use Amazon RDS for MySQL with Multi-AZ enabled.

This option provides better database availability with RDS Multi-AZ, but the consumer layer still relies on a single additional EC2 instance, which isn't as resilient as an Auto Scaling group.

- ☒ Use Amazon MQ with active/standby brokers configured across two Availability Zones. Add an Auto Scaling group for the consumer EC2 instances across two Availability Zones. Use Amazon RDS for MySQL with Multi-AZ enabled.

This option leverages managed services like Amazon MQ and RDS Multi-AZ for high availability and simplifies operations. The Auto Scaling group ensures the consumer layer is resilient and can scale

automatically based on demand.

Overall Explanation:

The architecture using Amazon MQ with active/standby brokers, an Auto Scaling group for consumer EC2 instances across two Availability Zones, and Amazon RDS for MySQL with Multi-AZ enabled offers the highest availability with low operational complexity. It leverages managed services for automated failover and scaling across all tiers (queue, processing, and database), maximizing fault tolerance and minimizing manual intervention. This makes option D the correct choice.

[View Discussions](#)[Ask AI](#)

Question 36

Not Attempted

[Compute](#)

A company hosts a containerized web application on a fleet of on-premises servers that process incoming requests. The number of requests is growing quickly, and the on-premises servers are struggling to handle the load. The company wants to migrate the application to AWS with minimal code changes and development effort. Which solution will meet these requirements with the LEAST operational overhead?

- ☒ Use AWS Fargate on Amazon Elastic Container Service (Amazon ECS) to run the containerized web application with Service Auto Scaling. Use an Application Load Balancer to distribute the incoming requests.
Fargate eliminates the need to manage servers, provides automatic scaling, and integrates with an Application Load Balancer for traffic distribution, all with minimal code changes.
- ☐ Use two Amazon EC2 instances to host the containerized web application. Use an Application Load Balancer to distribute the incoming requests.
This option requires manual management of EC2 instances, leading to higher operational overhead compared to serverless solutions like Fargate.
- ☐ Use AWS Lambda with new code that uses one of the supported languages. Create multiple Lambda functions to support the load. Use Amazon API Gateway as an entry point to the Lambda functions.
This option involves significant code changes and potentially restructuring the application, increasing development effort and operational complexity.

- ⊗ Use a high performance computing (HPC) solution such as AWS ParallelCluster to establish an HPC cluster that can process the incoming requests at the appropriate scale.

ParallelCluster is not suitable for a standard web application and introduces significant setup and configuration overhead, making it less efficient and more complex.

Overall Explanation:

The optimal solution is to use AWS Fargate on Amazon ECS with Service Auto Scaling and an Application Load Balancer. Fargate offers a serverless environment, minimizing operational overhead by removing the need to manage servers. It automatically scales the application based on demand. Using ECS provides a managed orchestration layer, and the Application Load Balancer handles traffic distribution efficiently. This approach requires minimal code changes, enabling a seamless migration to AWS with reduced development effort.

 View Discussions

 Ask AI

Question 37

Not Attempted

Storage



A company uses 50 TB of data for reporting and wants to migrate it from on-premises to AWS. A custom application in the company's data center runs a weekly data transformation job. The company plans to pause the application until the data transfer is complete and needs to begin the transfer process as soon as possible. The data center has no available network bandwidth for additional workloads. A solutions architect must transfer the data and configure the transformation job to continue running in the AWS Cloud with the LEAST operational overhead. Which solution meets these requirements?

- ⊗ Use AWS DataSync to move the data. Create a custom transformation job by using AWS Glue.

AWS DataSync requires available network bandwidth, which is not available in this scenario.

- ⊗ Order an AWS Snowcone device to move the data. Deploy the transformation application to the device.

AWS Snowcone has a limited storage capacity (14 TB SSD) that is insufficient for the 50 TB data transfer.

- ☒ Order an AWS Snowball Edge Storage Optimized device. Copy the data to the device. Create a custom transformation job by using AWS Glue.

This option uses Snowball Edge for data transfer without network bandwidth requirements and leverages AWS Glue, a serverless ETL service, for transformation, minimizing operational overhead.

- ☐ Order an AWS Snowball Edge Storage Optimized device that includes Amazon EC2 compute. Copy the data to the device. Create a new EC2 instance on AWS to run the transformation application.

While Snowball Edge with EC2 can transfer the data and run the transformation, managing an EC2 instance incurs more operational overhead compared to using AWS Glue.

Overall Explanation:

The most suitable solution is to use AWS Snowball Edge Storage Optimized to transfer the 50 TB of data, as there is no network bandwidth available. Using AWS Glue for the transformation job provides a serverless, low-overhead solution compared to managing an EC2 instance. Therefore, option C is the correct answer.

 View Discussions

 Ask AI

Question 38

Not Attempted

Compute



A company has developed an image analysis application where users upload photos and apply photo frames. Users upload images and metadata specifying the desired frames. Currently, the application runs on a single EC2 instance with DynamoDB for metadata storage. As the application's popularity grows, the company anticipates significant fluctuations in concurrent users based on the time of day and day of the week. To accommodate this growth and varying load, the company needs a scalable solution. Which solution meets these requirements most effectively?

- ☐ Use AWS Lambda to process the photos. Store the photos and metadata in DynamoDB.
Storing photos directly in DynamoDB is not recommended due to DynamoDB's limitations on item size and its unsuitability for storing large binary data like images.
- ☐ Use Amazon Kinesis Data Firehose to process the photos and to store the photos and metadata.

Kinesis Data Firehose is designed for real-time streaming data ingestion, not ideal for general image processing and storage.

- ✔ Use AWS Lambda to process the photos. Store the photos in Amazon S3. Retain DynamoDB to store the metadata.

This is the most suitable option. Lambda provides serverless compute for processing images, S3 offers scalable and durable storage for the images, and DynamoDB efficiently handles metadata storage.

- ✘ Increase the number of EC2 instances to three. Use Provisioned IOPS SSD (io2) Amazon Elastic Block Store (Amazon EBS) volumes to store the photos and metadata.

Scaling EC2 instances manually and using EBS volumes doesn't offer the same level of automated scaling and cost-effectiveness as a serverless solution like Lambda and S3.

Overall Explanation:

The most scalable and cost-effective solution is to use AWS Lambda to process images, store the images in Amazon S3, and retain DynamoDB for metadata. Lambda auto-scales based on demand, S3 provides virtually unlimited storage, and DynamoDB provides efficient querying of metadata. Storing images directly in DynamoDB is not recommended, and Kinesis is better suited for streaming data. Manually scaling EC2 instances is less efficient and scalable.

 View Discussions

 Ask AI

Question 39

Not Attempted

Networking



A medical records company hosts an application on Amazon EC2 instances within a VPC. The application processes customer data files stored in Amazon S3. Currently, the EC2 instances, located in public subnets, access S3 over the internet. Due to new compliance requirements, the network traffic for file transfers must now traverse a private route and avoid the public internet. The EC2 instances do not require any other internet access. Which network architecture change should a solutions architect recommend?

- ✘ Create a NAT gateway. Configure the route table for the public subnets to route traffic to Amazon S3 through the NAT gateway.

A NAT gateway allows instances in private subnets to connect to the internet, but doesn't ensure private S3 access. Traffic would still traverse the internet.

- ⊗ Configure the security group for the EC2 instances to restrict outbound traffic so that only traffic to the S3 prefix list is permitted.

Security groups control inbound and outbound traffic, but do not establish a private connection to S3. Traffic would still use the public internet.

- ✔ Move the EC2 instances to private subnets. Create a VPC endpoint for Amazon S3, and associate the endpoint with the route table for the private subnets.

This solution provides a private route to S3. Moving EC2 instances to private subnets and using a VPC endpoint ensures that traffic to S3 does not traverse the internet.

- ⊗ Remove the internet gateway from the VPC. Set up an AWS Direct Connect connection, and route traffic to Amazon S3 over the Direct Connect connection.

Direct Connect is typically used for hybrid environments to connect on-premise infrastructure. It's an overkill for simply routing traffic within the VPC to S3 privately.

Overall Explanation:

The most effective solution is to move the EC2 instances to private subnets and create a VPC endpoint for S3. The VPC endpoint ensures that traffic between the EC2 instances and S3 stays within the AWS network, avoiding the public internet and meeting the compliance requirements. A NAT Gateway still requires access to the internet. Direct connect is overkill for this scenario.

 View Discussions

 Ask AI

Question 40

Not Attempted

Serverless



A company currently hosts its corporate website on a traditional content management system (CMS). The patching and maintenance overhead associated with the CMS are proving to be burdensome. As part of a website redesign, the company seeks a more streamlined solution. The website is updated approximately four times per year, and dynamic content is not a requirement. The primary goals for the new solution are high scalability, enhanced security, and minimal operational overhead. Which combination of services will best meet these requirements?

- ✔ Configure Amazon CloudFront in front of the website to leverage its HTTPS functionality.

CloudFront can be configured to enforce HTTPS, providing enhanced security and leveraging its globally distributed network for scalability. This reduces operational overhead by offloading SSL/TLS management.

- ☐ Deploy an AWS WAF web ACL in front of the website to provide HTTPS functionality.
AWS WAF primarily protects against web exploits and vulnerabilities; it does not provide HTTPS functionality directly. While it can protect HTTPS traffic, it doesn't fulfill the HTTPS requirement.
- ☐ Create and deploy an AWS Lambda function to manage and serve the website content.
While Lambda can serve web content, it typically involves more complexity for serving static websites compared to S3 static website hosting and is more suited for dynamic content generation. Lambda also needs other AWS services like API Gateway to work with a website, increasing operational overhead.
- ☒ Create the new website and an Amazon S3 bucket. Deploy the website to the S3 bucket with static website hosting enabled.
Amazon S3 with static website hosting offers high scalability, requires minimal maintenance, and is a cost-effective solution for serving static website content. It automatically scales to meet increased demand.
- ☐ Create the new website. Deploy the website by using an Auto Scaling group of Amazon EC2 instances behind an Application Load Balancer.
Using EC2 instances with Auto Scaling and an Application Load Balancer increases operational overhead due to server management, patching, and scaling configurations. This solution is more suitable for dynamic content and applications requiring server-side processing.

Overall Explanation:

The optimal solution involves utilizing Amazon CloudFront to provide HTTPS functionality and security along with Amazon S3 for hosting the static website. CloudFront ensures scalability and enhanced security through its content delivery network, while S3 provides a highly scalable and low-maintenance static website hosting solution. This combination minimizes operational overhead, aligning with the company's objectives. AWS WAF does not provide HTTPS functionality directly. Using AWS Lambda or an EC2-based solution with Auto Scaling increases operational overhead compared to S3 static website hosting.

[View Discussions](#)[Ask AI](#)

A company stores its application logs in an Amazon CloudWatch Logs log group. A new policy requires the company to store all application logs in Amazon OpenSearch Service (Amazon Elasticsearch Service) in near-real time with the LEAST operational overhead. Which solution meets these requirements?

- ✔ Configure a CloudWatch Logs subscription to stream the logs to Amazon OpenSearch Service (Amazon Elasticsearch Service).

This option leverages the native CloudWatch Logs subscription feature for streaming logs to OpenSearch Service, minimizing operational overhead. It automates the process and avoids the need for managing additional infrastructure or code.

- ✘ Create an AWS Lambda function. Use the log group to invoke the function to write the logs to Amazon OpenSearch Service (Amazon Elasticsearch Service).

While Lambda can be used, it introduces operational overhead related to managing and maintaining the function, including scaling and error handling.

- ✘ Create an Amazon Kinesis Data Firehose delivery stream. Configure the log group as the delivery stream's source. Configure Amazon OpenSearch Service (Amazon Elasticsearch Service) as the delivery stream's destination.

Although Kinesis Data Firehose is a managed service, it requires more configuration and management compared to directly streaming via CloudWatch Logs subscription, adding to operational overhead.

- ✘ Install and configure Amazon Kinesis Agent on each application server to deliver the logs to Amazon Kinesis Data Streams. Configure Kinesis Data Streams to deliver the logs to Amazon OpenSearch Service (Amazon Elasticsearch Service).

This solution involves the most operational overhead because it requires installing and managing agents on each application server, as well as configuring and maintaining Kinesis Data Streams.

Overall Explanation:

Option A is the most efficient and requires the least operational overhead. CloudWatch Logs can be directly configured to stream logs to Amazon OpenSearch Service (Amazon Elasticsearch Service) using a subscription. This eliminates the need for additional services like Lambda or Kinesis, simplifying the setup and reducing management overhead. The other options involve more complex setups and increased maintenance, therefore, they are less optimal.

[View Discussions](#)[Ask AI](#)

Question 42

Not Attempted

Storage



A company is developing a web application that will run on Amazon EC2 instances spread across multiple Availability Zones. The application needs to store approximately 900 TB of text documents, and it's expected to handle periods of intense demand. A solutions architect needs to design a storage solution that can scale effectively to meet the application's demand at all times, while also minimizing overall costs. Which of the following storage solutions would be the MOST cost-effective for this scenario?

☐ Amazon Elastic Block Store (Amazon EBS)

Amazon EBS is block storage primarily designed for use with single EC2 instances. While it can be scaled, it's generally less cost-effective and doesn't scale as seamlessly as object storage solutions for large datasets across multiple instances.

☐ Amazon Elastic File System (Amazon EFS)

Amazon EFS is a scalable file storage service, and while it can handle the size requirements, it's generally more expensive than Amazon S3, especially for a large dataset like 900 TB.

☐ Amazon OpenSearch Service (Amazon Elasticsearch Service)

Amazon OpenSearch Service is a search and analytics service, not a primary storage solution. It is not designed to store large volumes of data long-term.

☒ Amazon S3

Amazon S3 is a highly scalable, durable, and cost-effective object storage service. It's an ideal solution for storing large amounts of unstructured data like text documents and can easily scale to meet the demands of a web application. S3 offers various storage classes to further optimize cost based on access frequency.

Overall Explanation:

Amazon S3 provides the most cost-effective solution for storing a large volume of text documents (900 TB) required by the web application. It offers scalability, durability, and cost optimization. EBS is not ideal for large scale object storage. EFS can be more expensive than S3 for this purpose. OpenSearch is for search and analytics and not designed to be a primary data store.

[View Discussions](#)[Ask AI](#)

Question 43

Not Attempted

Security



A global company is using Amazon API Gateway to design REST APIs for its loyalty club users in the us-east-1 Region and the ap-southeast-2 Region. A solutions architect must design a solution to protect these API Gateway managed REST APIs across multiple accounts from SQL injection and cross-site scripting attacks. What solution will meet these requirements with the LEAST amount of administrative effort?

- ☐ Set up AWS WAF in both Regions. Associate Regional web ACLs with an API stage.
This option requires individual WAF configuration in each region and association with API stages, increasing administrative effort.
- ☒ Set up AWS Firewall Manager in both Regions. Centrally configure AWS WAF rules.
AWS Firewall Manager provides centralized management of WAF rules across multiple accounts and regions, reducing administrative overhead by allowing for consistent security policies.
- ☐ Set up AWS Shield in both Regions. Associate Regional web ACLs with an API stage.
AWS Shield primarily protects against DDoS attacks, not SQL injection or cross-site scripting attacks.
- ☐ Set up AWS Shield in one of the Regions. Associate Regional web ACLs with an API stage.
AWS Shield primarily protects against DDoS attacks, not SQL injection or cross-site scripting attacks. Furthermore, setting it up in only one region doesn't meet the requirement of protecting both regions.

Overall Explanation:

AWS Firewall Manager centrally manages AWS WAF rules across multiple accounts and regions. This approach allows for the enforcement of consistent security policies across all API Gateway APIs, protecting them from SQL injection and cross-site scripting attacks with the least administrative effort. AWS WAF protects against web attacks using SQL code injection and cross-site scripting. AWS Firewall Manager simplifies your administration and maintenance tasks across multiple accounts and resources for WAF protections.

[View Discussions](#)[Ask AI](#)

Question 44

Not Attempted

Networking



A global company hosts its web application on Amazon EC2 instances behind an Application Load Balancer (ALB). The web application serves both static and dynamic data. Static data is stored in an Amazon S3 bucket. The company wants to improve performance and reduce latency for both types of data. The company uses its own domain name, registered with Amazon Route 53. Which solution will meet these requirements?

- ☒ Create an Amazon CloudFront distribution that has the S3 bucket and the ALB as origins. Configure Route 53 to route traffic to the CloudFront distribution.

This is the correct answer. CloudFront can serve both static content from S3 and dynamic content proxied from the ALB, improving performance and reducing latency. Route 53 directs traffic to the CloudFront distribution, ensuring efficient content delivery.
- ☐ Create an Amazon CloudFront distribution that has the ALB as an origin. Create an AWS Global Accelerator standard accelerator that has the S3 bucket as an endpoint. Configure Route 53 to route traffic to the CloudFront distribution.

This is incorrect. AWS Global Accelerator standard accelerator doesn't support S3 endpoints. Moreover, the ALB is already handling traffic, making Global Accelerator for S3 redundant.
- ☐ Create an Amazon CloudFront distribution that has the S3 bucket as an origin. Create an AWS Global Accelerator standard accelerator that has the ALB and the CloudFront distribution as endpoints. Create a custom domain name that points to the accelerator DNS name. Use the custom domain name as an endpoint for the web application.

This is incorrect. Global Accelerator cannot have CloudFront as an endpoint. It also suggests creating a new domain name, which is unnecessary as the company already has one.
- ☐ Create an Amazon CloudFront distribution that has the ALB as an origin. Create an AWS Global Accelerator standard accelerator that has the S3 bucket as an endpoint. Create two domain names. Point one domain name to the CloudFront DNS name for dynamic content. Point the other domain name to the accelerator DNS name for static content. Use the domain names as endpoints for the web application.

This is incorrect. Global Accelerator does not support S3 endpoints. Creating two domain names and managing traffic across them adds unnecessary complexity.

Overall Explanation:

The optimal solution is to use Amazon CloudFront with both the S3 bucket (for static data) and the ALB (for dynamic data) as origins. This allows CloudFront to cache static content

and accelerate dynamic content delivery. AWS Global Accelerator is not appropriate for this use case, and creating separate domain names would add unnecessary complexity. Option A effectively leverages CloudFront's capabilities to enhance performance and reduce latency for both static and dynamic content.

[View Discussions](#)[Ask AI](#)

Question 45

Not Attempted

Networking



A company has implemented a self-managed DNS solution on three Amazon EC2 instances behind a Network Load Balancer (NLB) in the us-west-2 Region. Most of the company's users are located in the United States and Europe. The company wants to improve the performance and availability of the solution. The company launches and configures three EC2 instances in the eu-west-1 Region and adds the EC2 instances as targets for a new NLB. Which solution can the company use to route traffic to all the EC2 instances?

- ☐ Create an Amazon Route 53 geolocation routing policy to route requests to one of the two NLBs. Create an Amazon CloudFront distribution. Use the Route 53 record as the distribution's origin.

This option isn't optimal because a self-managed DNS solution typically doesn't integrate directly with Route 53 for routing. Also, CloudFront is primarily a CDN and not meant for DNS routing.

- ☒ Create a standard accelerator in AWS Global Accelerator. Create endpoint groups in us-west-2 and eu-west-1. Add the two NLBs as endpoints for the endpoint groups.

This is the best solution. AWS Global Accelerator provides static IP addresses and routes traffic over the AWS global network, optimizing performance and availability. Endpoint groups in both regions allow for distributing traffic across both NLBs.

- ☐ Attach Elastic IP addresses to the six EC2 instances. Create an Amazon Route 53 geolocation routing policy to route requests to one of the six EC2 instances. Create an Amazon CloudFront distribution. Use the Route 53 record as the distribution's origin.

This option is not ideal. Directly attaching Elastic IPs to the instances and relying solely on Route 53 geolocation doesn't leverage the benefits of AWS's global network and is more complex to manage. Also, CloudFront is not for DNS.

- ☐ Replace the two NLBs with two Application Load Balancers (ALBs). Create an Amazon Route 53 latency routing policy to route requests to one of the two ALBs. Create an Amazon CloudFront distribution. Use the Route 53 record as the distribution's origin.

This option is not ideal for a self-managed DNS solution. Although ALBs and latency-based routing in Route 53 can improve performance, it doesn't leverage the global network optimizations provided by AWS Global Accelerator and it would be more effective if it was not a self-managed DNS

Overall Explanation:

AWS Global Accelerator offers a global solution by creating a standard accelerator and configuring endpoint groups in both regions. It can route traffic to all the EC2 instances across multiple regions by adding the two NLBs as endpoints, ensuring effective traffic distribution and leveraging the AWS global network for optimal performance and availability, making it ideal for a self-managed DNS solution.

[View Discussions](#)[Ask AI](#)

Question 46

Not Attempted

[Databases](#)

A company is running an online transaction processing (OLTP) workload on AWS. This workload uses an unencrypted Amazon RDS DB instance in a Multi-AZ deployment. Daily database snapshots are taken from this instance. What should a solutions architect do to ensure the database and snapshots are always encrypted moving forward?

- ☒ Encrypt a copy of the latest DB snapshot. Replace existing DB instance by restoring the encrypted snapshot.

This is the correct option. You can add encryption to an unencrypted DB instance by creating a snapshot, making an encrypted copy of it, and restoring a new DB instance from the encrypted snapshot. This also allows future snapshots to be encrypted.

- ☐ Create a new encrypted Amazon Elastic Block Store (Amazon EBS) volume and copy the snapshots to it. Enable encryption on the DB instance.

You cannot enable encryption on an existing RDS DB instance. EBS volumes are not used for RDS snapshots.

- ☐ Copy the snapshots and enable encryption using AWS Key Management Service (AWS KMS). Restore encrypted snapshot to an existing DB instance.

You cannot restore a DB snapshot to an existing DB instance. A new DB instance is created when restoring.

- ⊗ Copy the snapshots to an Amazon S3 bucket that is encrypted using server-side encryption with AWS Key Management Service (AWS KMS) managed keys (SSE-KMS).
Copying snapshots to an encrypted S3 bucket does not encrypt the RDS DB instance itself.

Overall Explanation:

The correct approach involves creating an encrypted copy of the latest DB snapshot and restoring a new DB instance from that encrypted snapshot. This ensures that the new DB instance is encrypted and that future snapshots will also be encrypted by default. Restoring to an existing DB instance is not possible, and simply encrypting the snapshots in S3 or EBS does not encrypt the database itself.

[View Discussions](#)[Ask AI](#)

Question 47

Not Attempted

Security



A company aims to establish a scalable key management infrastructure to support developers needing to encrypt data within their applications. Which action should a solutions architect take to minimize the operational overhead associated with managing encryption keys?

- ⊗ Use multi-factor authentication (MFA) to protect the encryption keys.
MFA enhances security but doesn't directly reduce the operational burden of key management itself.
- ✓ Use AWS Key Management Service (AWS KMS) to protect the encryption keys.
AWS KMS is a managed service that handles key generation, rotation, storage, and access control, significantly reducing operational overhead.
- ⊗ Use AWS Certificate Manager (ACM) to create, store, and assign the encryption keys.
AWS Certificate Manager is primarily for managing SSL/TLS certificates, not general-purpose encryption keys.
- ⊗ Use an IAM policy to limit the scope of users who have access permissions to protect the encryption keys.
IAM policies are important for access control, but do not address the underlying operational complexities of key management.

Overall Explanation:

AWS KMS is the best choice because it's a managed service that handles key management, rotation, and auditing. It integrates with other AWS services, scales automatically, offers fine-grained access controls, and uses HSMs for secure key storage, significantly reducing the operational burden on the company. Options A, C, and D provide security or access control, but do not provide a fully managed solution for key management.

[View Discussions](#)[Ask AI](#)

Question 48

Not Attempted

Security



A company hosts a dynamic web application on two Amazon EC2 instances, using the company's own SSL certificate installed on each instance for SSL termination. A recent traffic surge has caused the web servers to reach maximum compute capacity due to SSL encryption and decryption overhead. What should a solutions architect do to improve the application's performance?

- ☐ Create a new SSL certificate using AWS Certificate Manager (ACM). Install the ACM certificate on each instance.
Creating a new certificate doesn't address the SSL termination overhead on the EC2 instances.
- ☐ Create an Amazon S3 bucket. Migrate the SSL certificate to the S3 bucket. Configure the EC2 instances to reference the bucket for SSL termination.
Storing the certificate in S3 doesn't offload SSL processing from the EC2 instances and introduces complexity.
- ☐ Create another EC2 instance as a proxy server. Migrate the SSL certificate to the new instance and configure it to direct connections to the existing EC2 instances.
Adding another EC2 instance adds unnecessary complexity and management overhead without leveraging managed services for SSL termination and load balancing.
- ☒ Import the SSL certificate into AWS Certificate Manager (ACM). Create an Application Load Balancer with an HTTPS listener that uses the SSL certificate from ACM.
Using an Application Load Balancer (ALB) to handle SSL termination offloads the SSL processing from the EC2 instances, freeing up compute capacity and improving performance. ACM simplifies certificate management.

Overall Explanation:

The optimal solution is to import the existing SSL certificate into AWS Certificate Manager (ACM) and configure an Application Load Balancer (ALB) with an HTTPS listener to handle SSL termination. This offloads the CPU-intensive encryption/decryption tasks from the EC2 instances to the ALB, improving their overall performance. The EC2 instances then only receive unencrypted traffic.

[View Discussions](#)[Ask AI](#)

Question 49

Not Attempted

[Compute](#)

A company has a highly dynamic batch processing job that uses many Amazon EC2 instances to complete it. The job is stateless, can be started and stopped at any given time with no negative impact, and typically takes upwards of 60 minutes to complete. The company has asked a solutions architect to design a scalable and cost-effective solution that meets the requirements of the job. What should the solutions architect recommend?



Implement EC2 Spot Instances.

Spot Instances offer significant cost savings compared to On-Demand instances, and their interruptible nature is acceptable because the job is stateless and can be restarted without any negative impact. Spot instances are the most cost effective solution.



Purchase EC2 Reserved Instances.

Reserved Instances are better for stable workloads and do not offer significant cost savings for dynamic batch jobs.



Implement EC2 On-Demand Instances.

On-Demand Instances are more costly than Spot Instances and don't provide cost optimization for flexible workloads.



Implement the processing on AWS Lambda.

AWS Lambda has a timeout limit of 15 minutes, making it unsuitable for jobs that take upwards of 60 minutes to complete.

Overall Explanation:

EC2 Spot Instances are the most cost-effective solution for this scenario. The job's stateless nature and ability to be started/stopped at any time make it resilient to the potential interruptions of Spot Instances. Reserved Instances are not suitable for dynamic workloads, On-Demand instances are less cost-effective, and AWS Lambda's timeout limit precludes its use.

[View Discussions](#)[Ask AI](#)

Question 50

Not Attempted

Networking



A company runs its two-tier ecommerce website on AWS. The web tier consists of a load balancer that sends traffic to Amazon EC2 instances. The database tier uses an Amazon RDS DB instance. The EC2 instances and the RDS DB instance should not be exposed to the public internet. The EC2 instances require internet access to complete payment processing of orders through a third-party web service. The application must be highly available. Which combination of configuration options will meet these requirements? (Choose two.)

- ☒ Use an Auto Scaling group to launch the EC2 instances in private subnets. Deploy an RDS Multi-AZ DB instance in private subnets.
This option ensures that EC2 instances and RDS DB instance are not exposed to the public internet and provides high availability using Auto Scaling groups and RDS Multi-AZ.
- ☐ Configure a VPC with two private subnets and two NAT gateways across two Availability Zones. Deploy an Application Load Balancer in the private subnets.
While this option provides private subnets and NAT gateways, deploying the ALB in private subnets defeats the purpose of load balancing public traffic.
- ☐ Use an Auto Scaling group to launch the EC2 instances in public subnets across two Availability Zones. Deploy an RDS Multi-AZ DB instance in private subnets.
Launching EC2 instances in public subnets exposes them to the internet, which is against the requirements.
- ☐ Configure a VPC with one public subnet, one private subnet, and two NAT gateways across two Availability Zones. Deploy an Application Load Balancer in the public subnet.
This is not a suitable configuration for high availability as there is only one private subnet.

- ☒ Configure a VPC with two public subnets, two private subnets, and two NAT gateways across two Availability Zones. Deploy an Application Load Balancer in the public subnets.

This option combined with option A correctly configures the VPC to meet the requirements, the ALB handles external traffic, NAT gateways in the public subnets provide internet access for instances running in private subnets, and instances and database reside in private subnets.

Overall Explanation:

The correct options are A and the last option, which is E but displayed as D due to a typo. Option A ensures that the EC2 instances and RDS DB instance are not exposed to the public internet by placing them in private subnets. Auto Scaling and Multi-AZ deployment guarantee high availability. Option E (displayed as D) correctly configures the VPC with public subnets for the ALB and NAT Gateways, while maintaining the EC2 instances and RDS DB instance in private subnets, allowing the EC2 instances to access the internet for payment processing through the NAT Gateways. Two subnets across two Availability Zones will make the architecture highly available. NAT gateways in public subnets allow instances in private subnets to initiate outbound internet connections, but prevent inbound connections.

 View Discussions

 Ask AI

Question 51

Not Attempted

Storage



A solutions architect needs to implement a solution to reduce a company's storage costs. All the company's data is in the Amazon S3 Standard storage class. The company must keep all data for at least 25 years. Data from the most recent 2 years must be highly available and immediately retrievable. Which solution will meet these requirements?

- ☐ Set up an S3 Lifecycle policy to transition objects to S3 Glacier Deep Archive immediately.

Transitioning objects to S3 Glacier Deep Archive immediately would not fulfill the requirement of keeping the most recent 2 years of data highly available and immediately retrievable.

- ☒ Set up an S3 Lifecycle policy to transition objects to S3 Glacier Deep Archive after 2 years.

This is the correct solution. It keeps recent data readily available in S3 Standard for 2 years and then moves it to the cheaper Glacier Deep Archive for long-term storage.

- ⊗ Use S3 Intelligent-Tiering. Activate the archiving option to ensure that data is archived in S3 Glacier Deep Archive.

S3 Intelligent-Tiering with archiving might not guarantee that data remains in the standard tier for the full 2 years before archiving, potentially violating the immediate retrievability requirement.

- ⊗ Set up an S3 Lifecycle policy to transition objects to S3 One Zone-Infrequent Access (S3 One Zone-IA) immediately and to S3 Glacier Deep Archive after 2 years.

Transitioning objects to S3 One Zone-IA immediately might compromise availability requirements for the recent 2 years. S3 One Zone-IA doesn't offer the same level of availability as S3 Standard.

Overall Explanation:

The best solution is to set up an S3 Lifecycle policy to transition objects to S3 Glacier Deep Archive after 2 years. This ensures that the most recent data is highly available in S3 Standard for the required duration, and then cost-effectively archived for the long term in Glacier Deep Archive. S3 Intelligent-Tiering doesn't provide the necessary control over when data is moved to archive tiers, and S3 One Zone-IA affects data availability.

[View Discussions](#)[Ask AI](#)

Question 52

Not Attempted

Storage



A media company is evaluating the possibility of moving its systems to the AWS Cloud. The company needs at least 10 TB of storage with the maximum possible I/O performance for video processing, 300 TB of very durable storage for storing media content, and 900 TB of storage to meet requirements for archival media that is not in use anymore. Which set of services should a solutions architect recommend to meet these requirements?

- ⊗ Amazon EBS for maximum performance, Amazon S3 for durable data storage, and Amazon S3 Glacier for archival storage

Amazon EBS, while persistent, may not offer the maximum possible I/O performance compared to instance store. Instance store is physically attached and can offer higher performance. Additionally, it doesn't consider the temporary nature of the video processing.

- ⊗ Amazon EBS for maximum performance, Amazon EFS for durable data storage, and Amazon S3 Glacier for archival storage

Amazon EFS isn't as cost-effective or durable for large media libraries as S3, and EBS may not provide the maximum possible I/O performance.

- ⊗ Amazon EC2 instance store for maximum performance, Amazon EFS for durable data storage, and Amazon S3 for archival storage

While instance store provides high performance, Amazon EFS isn't suitable for durable data storage in this scenario due to cost and durability compared to S3, also S3 Glacier is better than S3 for archival storage.

- ✓ Amazon EC2 instance store for maximum performance, Amazon S3 for durable data storage, and Amazon S3 Glacier for archival storage

Amazon EC2 instance store offers the highest I/O performance for the video processing workload. Amazon S3 provides durable and scalable storage for media content. Amazon S3 Glacier offers cost-effective archival storage for infrequently accessed media.

Overall Explanation:

The correct solution is to use Amazon EC2 instance store for maximum I/O performance for video processing, Amazon S3 for durable storage of media content, and Amazon S3 Glacier for cost-effective archival storage. Instance store is ideal for temporary, high-performance workloads. S3 provides excellent durability and scalability for media content. Glacier is optimized for low-cost archival storage.

 View Discussions

 Ask AI

Question 53

Not Attempted

Compute



A company wants to run applications in containers in the AWS Cloud. These applications are stateless and can tolerate disruptions within the underlying infrastructure. The company needs a solution that minimizes cost and operational overhead. What should a solutions architect do to meet these requirements?

- ⊗ Use Spot Instances in an Amazon EC2 Auto Scaling group to run the application containers.

While this utilizes spot instances for cost savings, it requires more manual configuration and management compared to EKS managed node groups, potentially increasing operational overhead.

- ☒ Use Spot Instances in an Amazon Elastic Kubernetes Service (Amazon EKS) managed node group.

This is the best option. Spot Instances minimize cost, and EKS managed node groups minimize operational overhead by automating tasks like patching, scaling, and version upgrades.

- ☐ Use On-Demand Instances in an Amazon EC2 Auto Scaling group to run the application containers.

On-Demand Instances increase costs and EC2 Auto Scaling group will have higher operational overhead. This does not meet the requirements.

- ☐ Use On-Demand Instances in an Amazon Elastic Kubernetes Service (Amazon EKS) managed node group.

On-Demand Instances increase costs. This does not meet the requirements.

Overall Explanation:

The optimal solution is to use Spot Instances within an Amazon Elastic Kubernetes Service (Amazon EKS) managed node group. Spot Instances provide cost savings, and EKS managed node groups reduce operational overhead by managing the underlying infrastructure and Kubernetes cluster. Since the applications are stateless and tolerant to disruptions, Spot Instance interruptions are acceptable.

 View Discussions

 Ask AI

Question 54

Not Attempted

Compute



A company is running a multi-tier web application on-premises. The web application is containerized and runs on a number of Linux hosts connected to a PostgreSQL database that contains user records. The operational overhead of maintaining the infrastructure and capacity planning is limiting the company's growth. A solutions architect must improve the application's infrastructure. Which combination of actions should the solutions architect take to accomplish this? (Choose two.)

- ☒ Migrate the PostgreSQL database to Amazon Aurora.

Migrating to Aurora reduces operational overhead by leveraging a managed database service, providing scalability, automated backups, and high availability, reducing the need for manual database administration.

- ☐ **Migrate the web application to be hosted on Amazon EC2 instances.**
Migrating to EC2 instances still requires managing the underlying infrastructure, including operating system patching, scaling, and maintenance, which does not reduce operational overhead.
- ☐ **Set up an Amazon CloudFront distribution for the web application content.**
CloudFront improves content delivery and reduces latency but does not directly address the operational overhead and capacity planning challenges related to the application's infrastructure.
- ☐ **Set up Amazon ElastiCache between the web application and the PostgreSQL database.**
ElastiCache improves application performance by caching frequently accessed data but does not directly reduce operational overhead or address capacity planning limitations.
- ☒ **Migrate the web application to be hosted on AWS Fargate with Amazon Elastic Container Service (Amazon ECS).**
Migrating to Fargate eliminates the need to manage EC2 instances, simplifying deployment, improving resource utilization, and reducing operational overhead associated with managing the container infrastructure.

Overall Explanation:

Migrating the PostgreSQL database to Amazon Aurora and migrating the web application to AWS Fargate with Amazon ECS significantly reduces operational overhead. Aurora offers a managed database service, simplifying database administration, and Fargate removes the need to manage container infrastructure. These actions allow the company to focus on application development and growth rather than infrastructure management. Therefore, A and E are the correct answers.

[View Discussions](#)[Ask AI](#)

Question 55

Not Attempted

[Databases](#)

A company performs monthly maintenance on its AWS infrastructure. During these maintenance activities, the company needs to rotate the credentials for its Amazon RDS for MySQL databases across multiple AWS Regions. Which solution will meet these requirements with the LEAST operational overhead?

- ☒ **Store the credentials as secrets in AWS Secrets Manager. Use multi-Region secret replication for the required Regions. Configure Secrets Manager to rotate the secrets on**

a schedule.

AWS Secrets Manager supports encrypting credentials for RDS, DocumentDb, Redshift, other DBs and key/value secrets. It also supports multi-region replication and rotation based on a schedule, thus offering the least operational overhead.

- ⊗ Store the credentials as secrets in AWS Systems Manager by creating a secure string parameter. Use multi-Region secret replication for the required Regions. Configure Systems Manager to rotate the secrets on a schedule.

Secure string parameter in Systems Manager is incorrect because it only applies to Parameter Store. All data in AWS Secrets Manager is encrypted by default.

- ⊗ Store the credentials in an Amazon S3 bucket that has server-side encryption (SSE) enabled. Use Amazon EventBridge (Amazon CloudWatch Events) to invoke an AWS Lambda function to rotate the credentials.

Storing credentials in S3 would require manual implementation of rotation and replication, significantly increasing operational overhead. S3 also doesn't offer a built-in secret rotation feature.

- ⊗ Encrypt the credentials as secrets by using AWS Key Management Service (AWS KMS) multi-Region customer managed keys. Store the secrets in an Amazon DynamoDB global table. Use an AWS Lambda function to retrieve the secrets from DynamoDB. Use the RDS API to rotate the secrets.

Using KMS, DynamoDB, and Lambda involves many more steps than using Secrets Manager, leading to higher operational overhead.

Overall Explanation:

AWS Secrets Manager is a secrets management service designed for storing, managing, and rotating secrets like database credentials across multiple regions. The multi-region replication and scheduled rotation capabilities minimize operational overhead, making it the most suitable solution. Using other services would necessitate more manual configuration and management.

[View Discussions](#)[Ask AI](#)

Question 56

Not Attempted

[Compute](#)

An application runs on Amazon EC2 instances across multiple Availability Zones. The instances run in an Amazon EC2 Auto Scaling group behind an Application Load Balancer. The application

performs best when the CPU utilization of the EC2 instances is consistently at or near 40%.

What should a solutions architect do to maintain the desired performance across all instances in the group?

- ☐ Use a simple scaling policy to dynamically scale the Auto Scaling group.
A simple scaling policy only responds to breaching thresholds and might not effectively maintain a target CPU utilization.
- ☒ Use a target tracking policy to dynamically scale the Auto Scaling group.
A target tracking policy allows the Auto Scaling group to automatically adjust the number of EC2 instances based on a target value for a metric, in this case, CPU utilization.
- ☐ Use an AWS Lambda function to update the desired Auto Scaling group capacity.
While Lambda could be used, it requires custom scripting and may not provide the same level of automation and responsiveness as a target tracking policy.
- ☐ Use scheduled scaling actions to scale up and scale down the Auto Scaling group.
Scheduled scaling actions are not suitable for maintaining the desired performance in real-time based on actual CPU utilization; it's based on predefined times.

Overall Explanation:

The correct solution is to use a target tracking policy. This policy allows defining a specific target metric value to maintain (40% CPU utilization in this case). Auto Scaling will automatically add or remove instances to keep utilization at the target level, without manual intervention. Target tracking policies are the native Auto Scaling feature designed to maintain a metric at a target value.

[View Discussions](#)[Ask AI](#)

Question 57

Not Attempted

Security



A company is developing a file-sharing application using an Amazon S3 bucket for storage and Amazon CloudFront for content delivery. To enhance security, the company wants to ensure that files are only accessible through the CloudFront distribution and not directly via the S3 URL. What is the MOST secure and recommended solution to meet these requirements?

- ⊗ Write individual policies for each S3 object granting read permission to only CloudFront.
Managing individual policies for each S3 object is complex and difficult to scale, especially with numerous objects.
- ⊗ Create an IAM user, grant read permission to the S3 bucket, and associate this user with CloudFront.
Using an IAM user exposes credentials and does not effectively prevent direct S3 URL access.
- ⊗ Write an S3 bucket policy assigning the CloudFront distribution ID as the Principal and the S3 bucket as the ARN.
While this restricts access, using an Origin Access Identity (OAI) or Origin Access Control (OAC) is the preferred method.
- ✔ Create an origin access identity (OAI) or Origin Access Control (OAC), assign it to the CloudFront distribution, and grant the OAI/OAC read permission to the S3 bucket.
Creating an OAI/OAC provides secure access between CloudFront and S3, preventing direct S3 URL access and ensuring only CloudFront can access the objects.

Overall Explanation:

The MOST secure and recommended approach is to use an Origin Access Identity (OAI) or Origin Access Control (OAC). By creating an OAI/OAC and associating it with the CloudFront distribution, you can control access to the S3 bucket, preventing direct access via the S3 URL. The OAI/OAC has permission to access objects, while direct access is denied to users. OAC is recommended over OAI since OAC supports more features like SSE-KMS.

[View Discussions](#)[Ask AI](#)

Question 58

Not Attempted

Serverless



A company's website provides users with downloadable historical performance reports. The website needs a solution that will scale to meet the company's website demands globally. The solution should be cost-effective, limit the provisioning of infrastructure resources, and provide the fastest possible response time. Which combination should a solutions architect recommend to meet these requirements?



Amazon CloudFront and Amazon S3

Amazon CloudFront is a CDN that caches content at edge locations globally, reducing latency. Amazon S3 provides cost-effective and scalable storage for static content.



AWS Lambda and Amazon DynamoDB

AWS Lambda and Amazon DynamoDB are not suitable for serving downloadable files globally. Lambda is more suited for event-driven compute, and DynamoDB is a NoSQL database.



Application Load Balancer with Amazon EC2 Auto Scaling

This option requires provisioning and managing more infrastructure compared to CloudFront and S3. It might not provide the same global scalability and low latency.



Amazon Route 53 with internal Application Load Balancers

Amazon Route 53 is a DNS service but doesn't provide caching or content delivery capabilities. Internal Application Load Balancers don't address global scalability.

Overall Explanation:

Amazon CloudFront and Amazon S3 offer a scalable and cost-effective solution that limits infrastructure provisioning and provides the fastest response time for globally distributing static content such as historical performance reports. CloudFront caches the reports at edge locations, reducing latency for users worldwide. S3 provides the underlying storage.

[View Discussions](#)[Ask AI](#)

Question 59

Not Attempted

[Databases](#)

A company wants to migrate its on-premises Oracle database to AWS and upgrade it to the latest version. They require a disaster recovery (DR) solution with minimal operational overhead. Critically, they also need to maintain access to the underlying operating system of the database server. Which solution best meets these requirements?



Migrate the Oracle database to an Amazon EC2 instance. Set up database replication to a different AWS Region.

This option provides OS access but significantly increases operational overhead due to managing the database and replication yourself. Setting up and managing replication manually adds complexity.

- ⊗ Migrate the Oracle database to Amazon RDS for Oracle. Activate Cross-Region automated backups to replicate the snapshots to another AWS Region.
- RDS for Oracle reduces operational overhead, but it does **not** provide access to the underlying operating system, which is a key requirement.
- ✔ Migrate the Oracle database to Amazon RDS Custom for Oracle. Create a read replica for the database in another AWS Region.
- RDS Custom for Oracle provides access to the underlying OS and reduces operational overhead compared to EC2. Creating a read replica in another region can serve as a DR solution. RDS Custom supports creating read replicas, although cross-region replicas have limitations.
- ⊗ Migrate the Oracle database to Amazon RDS for Oracle. Create a standby database in another Availability Zone.
- RDS for Oracle reduces operational overhead and provides high availability through standby databases in another AZ, but it doesn't provide access to the underlying operating system. This option also focuses on HA within a single region rather than DR in another region.

Overall Explanation:

Amazon RDS Custom for Oracle is the best choice because it offers a balance between managed service convenience, access to the underlying OS, and disaster recovery capabilities with cross-region read replicas. While not a full DR solution in the traditional sense, a read replica can be used for DR purposes. RDS Custom allows OS access needed for specific customizations and operational requirements, while minimizing operational overhead through AWS-managed services. The other options either lack OS access or significantly increase operational overhead.

[View Discussions](#)[Ask AI](#)

Question 60

Not Attempted

[Serverless](#)

A company wants to migrate its application to a serverless architecture. The solution must analyze both existing and newly generated data using SQL. The company currently stores its data in an Amazon S3 bucket. Data requires encryption at rest, and business requirements dictate that the data must be replicated to a different AWS Region for disaster recovery. Which solution will meet these requirements while minimizing operational overhead?

- ⊗ Create a new S3 bucket. Load the data into the new S3 bucket. Use S3 Cross-Region Replication (CRR) to replicate encrypted objects to an S3 bucket in another Region. Use server-side encryption with AWS KMS multi-Region keys (SSE-KMS). Use Amazon Athena to query the data.

This option involves creating a new bucket and using SSE-KMS, which provides enhanced key management but adds operational overhead. While it ensures encryption and replication, the creation of a new bucket and loading existing data increases complexity.

- ⊗ Create a new S3 bucket. Load the data into the new S3 bucket. Use S3 Cross-Region Replication (CRR) to replicate encrypted objects to an S3 bucket in another Region. Use server-side encryption with AWS KMS multi-Region keys (SSE-KMS). Use Amazon RDS to query the data.

This option uses RDS for querying, which is not suitable for analyzing data directly in S3. RDS is a relational database, while Athena is better suited for querying data in S3. It also includes the overhead of creating and loading data into a new bucket and using SSE-KMS.

- ✔ Load the data into the existing S3 bucket. Use S3 Cross-Region Replication (CRR) to replicate encrypted objects to an S3 bucket in another Region. Use server-side encryption with Amazon S3 managed encryption keys (SSE-S3). Use Amazon Athena to query the data.

This is the most efficient option. It leverages the existing bucket, uses SSE-S3 for encryption (which has less overhead than SSE-KMS), and uses Athena for serverless querying of data in S3. CRR handles the replication to another region. By default S3 now encrypts the objects with SSE-S3.

- ⊗ Load the data into the existing S3 bucket. Use S3 Cross-Region Replication (CRR) to replicate encrypted objects to an S3 bucket in another Region. Use server-side encryption with Amazon S3 managed encryption keys (SSE-S3). Use Amazon RDS to query the data.

This option uses RDS to query data residing in S3 which is not optimal. Athena is a serverless query service designed for data in S3. Using RDS introduces unnecessary operational overhead.

Overall Explanation:

The optimal solution is to utilize the existing S3 bucket with S3-managed encryption (SSE-S3) and Athena for querying. S3 Cross-Region Replication (CRR) handles the data replication to the secondary region. This approach minimizes operational overhead by avoiding the creation of new buckets and leverages serverless solutions. Athena is designed to query data directly from S3, making it the more efficient and less operationally intensive choice compared to using a relational database like RDS.

[View Discussions](#)[Ask AI](#)

Question 61

Not Attempted

Networking



A company hosts its workloads on AWS and needs to connect to a service provided by an external vendor. This service resides within the vendor's VPC. The security team mandates a private connection that is restricted solely to the target service, and the connection must originate exclusively from the company's VPC. Which solution satisfies these requirements?

- ☐ Create a VPC peering connection between the company's VPC and the provider's VPC. Update the route table to connect to the target service.
VPC peering allows communication between entire VPCs, potentially violating the requirement to restrict connectivity only to the target service.
- ☐ Ask the provider to create a virtual private gateway in its VPC. Use AWS PrivateLink to connect to the target service.
While PrivateLink is appropriate, the provider creating a VGW isn't the standard approach. This also might introduce unnecessary complexity or security concerns.
- ☐ Create a NAT gateway in a public subnet of the company's VPC. Update the route table to connect to the target service.
A NAT gateway facilitates outbound internet access, contradicting the requirement for a private connection.
- ☒ Ask the provider to create a VPC endpoint for the target service. Use AWS PrivateLink to connect to the target service.
This solution utilizes AWS PrivateLink and VPC endpoints, providing a private, secure, and restricted connection to the target service, initiated from the company's VPC.

Overall Explanation:

The correct answer is option D. AWS PrivateLink allows you to privately access services hosted on AWS, on-premises, or in other VPCs. By creating a VPC endpoint for the target service in the provider's VPC, and utilizing PrivateLink, the company establishes a secure and private connection that adheres to the security team's requirements. VPC peering allows general communication between VPCs, and a NAT Gateway uses public internet access.

[View Discussions](#)[Ask AI](#)

Question 62

Not Attempted

Databases



A company is migrating its on-premises PostgreSQL database to Amazon Aurora PostgreSQL. The on-premises database must remain online and accessible during the migration. The Aurora database must remain synchronized with the on-premises database. Which combination of actions must a solutions architect take to meet these requirements?

☒ Create an ongoing replication task.

Creating an ongoing replication task ensures continuous data synchronization between the on-premises PostgreSQL database and the Aurora PostgreSQL database, keeping the Aurora database up-to-date with the on-premises database changes.

☐ Create a database backup of the on-premises database.

Creating a database backup is helpful but does not ensure ongoing synchronization. It's a one-time snapshot.

☒ Create an AWS Database Migration Service (AWS DMS) replication server.

AWS DMS facilitates the migration and replication process. A replication server is required to run the replication tasks and manage the data flow between the on-premises and Aurora databases.

☐ Convert the database schema by using the AWS Schema Conversion Tool (AWS SCT).

AWS SCT is typically used when migrating between different database engines. Since the migration is from PostgreSQL to Aurora PostgreSQL (which is PostgreSQL-compatible), schema conversion is not required.

☐ Create an Amazon EventBridge (Amazon CloudWatch Events) rule to monitor the database synchronization.

Amazon EventBridge (Amazon CloudWatch Events) can monitor but does not handle migration or replication.

Overall Explanation:

To maintain an online on-premises database during migration and keep the Aurora database synchronized, you must create an ongoing replication task (Option A) to continuously replicate data and an AWS DMS replication server (Option C) to manage the replication process. The AWS SCT tool is not needed in this scenario as it's PostgreSQL to

Aurora PostgreSQL, so the schema should be compatible. A simple backup does not synchronize the data.

[View Discussions](#)[Ask AI](#)

Question 63

Not Attempted

Security



A company uses AWS Organizations to create dedicated AWS accounts for each business unit, managing each unit's account independently upon request. The root email recipient missed a notification sent to the root user email address of one account. To ensure future notifications are not missed and are limited to account administrators, which solution should the company implement?

- ☐ ☒ **Configure the company's email server to forward notification email messages sent to the AWS account root user email address to all users in the organization.**
This option floods all users with notifications, lacking granularity and not limiting notifications to administrators.
- ☒ **Configure all AWS account root user email addresses as distribution lists that go to a few administrators who can respond to alerts. Configure AWS account alternate contacts in the AWS Organizations console or programmatically.**
This option ensures notifications are monitored by administrators and allows for centralized management and appropriate responses. Using alternate contacts provides flexibility.
- ☐ **Configure all AWS account root user email messages to be sent to one administrator who is responsible for monitoring alerts and forwarding those alerts to the appropriate groups.**
This option introduces a single point of failure and potential delays in forwarding alerts to the appropriate groups.
- ☐ **Configure all existing AWS accounts and all newly created accounts to use the same root user email address. Configure AWS account alternate contacts in the AWS Organizations console or programmatically.**
Using the same root user email address for all accounts is not recommended and restricts customization for specific accounts. Each AWS account requires a unique root email address. Using the same root email is not a best practice for security reasons.

Overall Explanation:

Configuring each AWS account root user email address as a distribution list that goes to a group of administrators, along with setting up alternate contacts, ensures that notifications are not missed and are limited to account administrators. This approach provides centralized management, timely responses, and avoids single points of failure. Option A sends the notification to everyone, Option C has a single point of failure and Option D is impractical given each account has its unique root email.

[View Discussions](#)[Ask AI](#)

Question 64

Not Attempted

[Compute](#)

A company runs its e-commerce application on AWS. Every new order is published as a message in a RabbitMQ queue that runs on an Amazon EC2 instance in a single Availability Zone. These messages are processed by a different application that runs on a separate EC2 instance. This application stores the details in a PostgreSQL database on another EC2 instance. All the EC2 instances are in the same Availability Zone. The company needs to redesign its architecture to provide the highest availability with the least operational overhead. What should a solutions architect do to meet these requirements?

- ☐ Migrate the queue to a redundant pair (active/standby) of RabbitMQ instances on Amazon MQ. Create a Multi-AZ Auto Scaling group for EC2 instances that host the application. Create another Multi-AZ Auto Scaling group for EC2 instances that host the PostgreSQL database.

This option involves managing both the application and database on EC2 instances with Auto Scaling, increasing operational overhead. It also doesn't leverage managed database services.

- ☒ Migrate the queue to a redundant pair (active/standby) of RabbitMQ instances on Amazon MQ. Create a Multi-AZ Auto Scaling group for EC2 instances that host the application. Migrate the database to run on a Multi-AZ deployment of Amazon RDS for PostgreSQL.

This solution leverages Amazon MQ for highly available messaging, a Multi-AZ Auto Scaling group for the application, and Amazon RDS for a highly available and managed database. This provides the best balance of availability and reduced operational overhead.

- ☐ Create a Multi-AZ Auto Scaling group for EC2 instances that host the RabbitMQ queue. Create another Multi-AZ Auto Scaling group for EC2 instances that host the application. Migrate the database to run on a Multi-AZ deployment of Amazon RDS for PostgreSQL.

This option involves managing the RabbitMQ queue on EC2 instances, increasing operational overhead compared to using Amazon MQ.

- ⊗ Create a Multi-AZ Auto Scaling group for EC2 instances that host the RabbitMQ queue. Create another Multi-AZ Auto Scaling group for EC2 instances that host the application. Create a third Multi-AZ Auto Scaling group for EC2 instances that host the PostgreSQL database.

This option involves managing both the RabbitMQ queue and the database on EC2 instances with Auto Scaling, significantly increasing operational overhead. There is no managed database solution.

Overall Explanation:

The optimal solution is to migrate the RabbitMQ queue to Amazon MQ for high availability and simplified management. Simultaneously, implementing a Multi-AZ Auto Scaling group for the application EC2 instances enhances its availability. Migrating the database to Amazon RDS for PostgreSQL in a Multi-AZ configuration further ensures high availability and reduces operational overhead by leveraging a managed service. This approach balances availability with minimized operational effort.

[View Discussions](#)[Ask AI](#)

Question 65

Not Attempted

Serverless



A reporting team receives files each day in an Amazon S3 bucket. The reporting team manually reviews and copies the files from this initial S3 bucket to an analysis S3 bucket each day at the same time to use with Amazon QuickSight. Additional teams are starting to send more files in larger sizes to the initial S3 bucket. The reporting team wants to move the files automatically to the analysis S3 bucket as the files enter the initial S3 bucket. The reporting team also wants to use AWS Lambda functions to run pattern-matching code on the copied data. In addition, the reporting team wants to send the data files to a pipeline in Amazon SageMaker Pipelines. What should a solutions architect do to meet these requirements with the LEAST operational overhead?

- ⊗ Create a Lambda function to copy the files to the analysis S3 bucket. Create an S3 event notification for the analysis S3 bucket. Configure Lambda and SageMaker Pipelines as destinations of the event notification. Configure s3:ObjectCreated:Put as the event type.

This solution involves using Lambda to copy the files and S3 event notifications directly to Lambda and SageMaker. However, S3 event notifications cannot directly target SageMaker Pipelines.

- ⊗ Create a Lambda function to copy the files to the analysis S3 bucket. Configure the analysis S3 bucket to send event notifications to Amazon EventBridge (Amazon CloudWatch Events). Configure an ObjectCreated rule in EventBridge (CloudWatch Events). Configure Lambda and SageMaker Pipelines as targets for the rule.

This solution involves using Lambda to copy the files and leverages EventBridge to route events to Lambda and SageMaker. While EventBridge supports SageMaker as a target, copying the files with Lambda introduces operational overhead compared to S3 replication.

- ⊗ Configure S3 replication between the S3 buckets. Create an S3 event notification for the analysis S3 bucket. Configure Lambda and SageMaker Pipelines as destinations of the event notification. Configure s3:ObjectCreated:Put as the event type.

This solution utilizes S3 replication but attempts to use S3 event notifications directly to Lambda and SageMaker. S3 event notifications cannot directly target SageMaker Pipelines.

- ✓ Configure S3 replication between the S3 buckets. Configure the analysis S3 bucket to send event notifications to Amazon EventBridge (Amazon CloudWatch Events). Configure an ObjectCreated rule in EventBridge (CloudWatch Events). Configure Lambda and SageMaker Pipelines as targets for the rule.

This solution leverages S3 replication for efficient file copying. EventBridge enables routing events to both Lambda for pattern matching and SageMaker Pipelines. S3 replication provides automated file copying and EventBridge offers flexible event routing.

Overall Explanation:

The solution with the least operational overhead is to use S3 replication to copy files between buckets. S3 replication is more efficient than copying with Lambda. S3 event notifications can be sent to Amazon EventBridge. EventBridge can then be configured to send the data files to both a Lambda function and a pipeline in Amazon SageMaker Pipelines.

 View Discussions

 Ask AI

[← Back to Exam](#)



Marketing

[Affiliate Program](#)

[Unlimited Access](#)

[Business Unlimited Access](#)

Support

[Contact Us](#)

team@itcertifications.io

Company

[Udemy Alternative](#)

[Whizlabs Alternative](#)

[TutorialsDojo Alternative](#)

Legal

[Terms of service](#)

[Privacy policy](#)

[Refund policy](#)

[Shipping policy](#)



© 2026, ITCertifications Technologies. All rights reserved.